Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

Intelligent Adaptive Service Grid Architecture Design and Performance Optimization for Future Networks

YuFeng Hu

YuFeng.Hu227@outlook.com

MinMin Liu

Research Article

Keywords: microservices, service grid architecture, intelligent routing, dynamic traffic management, network resilience

Posted Date: November 4th, 2024

DOI: https://doi.org/10.21203/rs.3.rs-5289129/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

Intelligent Adaptive Service Grid Architecture Design and Performance Optimization for Future Networks

a. Yufeng Hu College of Aeronautics and Astronautics, University of Electronic Science and Technology

YuFeng.Hu227@outlook.com

b. Minmin Liu College of Aeronautics and Astronautics, University of Electronic Science and Technology.

liumin_uestc@uestc.edu.cn

Abstract

With the development of future networks, the popularization of technologies such as 5G and the Internet of Things (IoT) has led to new characteristics of network environments with high dynamics, low latency, and complexity. Traditional service grid architectures have limitations in coping with the dynamics and complex traffic management of future networks, such as increased latency and inflexible traffic management. This study aims to design an intelligent and adaptive service grid architecture to optimize the performance to cope with the new challenges in future networks.

To address the network routing optimization problem, this study uses deep Q-learning algorithms to achieve intelligent routing, which effectively reduces the network delay and packet loss rate. To address the complexity of traffic management in large-scale distributed systems, a dynamic traffic management module combining convolutional neural networks and long and short-term memory networks is designed to improve the accuracy of traffic prediction. To enhance network adaptability and fault recovery, the study introduces a network resilience enhancement module, which ensures service continuity under high load and fault conditions. In addition, efficient transmission and low resource consumption under multiple network protocols are realized by the design of protocol adaptive module.

Experimental evaluation of the entire optimized architecture shows that this intelligent service grid architecture exhibits excellent performance in future network environments. The intelligent routing module effectively reduces network delay and packet loss, the dynamic traffic management module improves the accuracy of traffic prediction, the network resilience enhancement module ensures service continuity under high load and fault conditions, and the protocol adaptation module demonstrates efficient transmission and low resource consumption under multiple network protocols. Through the optimal design of the service grid architecture and the introduction of intelligent technologies, the performance and adaptability of service grids in future network environments are successfully enhanced, and the deficiencies faced by traditional service grids in dynamic network environments are addressed. This research provides important technical support and development direction for intelligent service grids in future networks.

Keywords: microservices; service grid architecture; intelligent routing; dynamic traffic management; network resilience

1. Introduction

1.1 Background

With the continuous development of cloud computing technology, cloud-native architecture has become the preferred approach for building and deploying applications in modern organizations [1]. The core features of cloud-native architectures are their high scalability, elasticity, and flexibility, and microservice architectures are typical representatives of this paradigm. By splitting an application into a series of independent services, microservices architecture can significantly improve the maintainability and scalability of a system. However, as microservices grow in size and complexity, the communication and management of services becomes increasingly complex. In order to solve these problems, Service Mesh has emerged [2]. Service Mesh decouples application business logic from infrastructure management by introducing a transparent infrastructure layer that manages inter-service communication, security policies and traffic control. It consists of a data plane (handling requests) and a control plane (managing configurations and policies), which improves the management efficiency and reliability of microservice architectures.

Future networks will be characterized by high dynamics, ultra-low latency, and ultra-large-scale connectivity. 5G promotes massive IoT device connectivity and smart application development by virtue of high bandwidth, low latency, and large number of connections. 6G, which is still in the R&D stage and is expected to be commercially available around 2030, will further enhance communication capabilities, support higher frequency bands, wider connectivity range and stronger AI integration, and the theoretical data transmission rate is expected to reach more than 1Tbps. The rapid development of IoT has given rise to applications such as smart homes and smart cities, and is driving the popularization of edge computing to enable local data processing and analysis, improve efficiency and reduce the burden on the cloud.

To cope with higher dynamics, lower latency requirements and more complex traffic management in future networks, service grid architectures need to be optimized and innovated in several ways. This includes improving the performance and flexibility of the control and data planes, introducing intelligent routing and load balancing mechanisms, and enhancing the adaptability to different network scenarios. In this literature [3], the challenges faced when dealing with high dynamics, low latency requirements, and complex traffic management are described in detail, specifically, then, including:

i. Challenges of a highly dynamic network environment

In highly dynamic network environments, service instances and network conditions may change frequently, e.g., service scaling up and scaling down, changes in network topology, addition of new services, or emergence of failed nodes. These changes pose serious challenges to the Service Grid architecture:

a. Real-Time Configuration and Policies: In dynamic environments, service grids need to be able to quickly sense and adapt to changes in the environment. However, current control planes often struggle to update policies and configurations in real-time, resulting in lagging configurations that cannot effectively respond to rapid changes in the network.
b. Complexity of state synchronization: due to the dynamic changes of service instances, service grids need to maintain state consistency in a distributed environment. This

becomes more complex in environments with frequent node changes, which can lead to data inconsistencies or service outages.

c. Resource Scheduling Flexibility: In highly dynamic environments, how to dynamically schedule service instances across multiple compute resources (e.g., cloud and edge) to optimally utilize network and compute resources remains a major challenge for Service Grids.

ii. Challenges of low latency requirements

With the rise of real-time applications (e.g., online gaming, video conferencing, autonomous driving, etc.), low latency has become a critical requirement in Service Grid architectures. However, current Service Grid architectures face the following challenges in achieving ultra-low latency:

a. Data-plane forwarding performance: Service grids typically handle all network requests through data-plane proxies, but these proxies may introduce additional network overhead and increase the latency of request processing. This can be a bottleneck in applications that require ultra-low latency.

b. Multi-hop latency of links: Service calls in a Service Grid may need to span multiple service instances, which can lead to the accumulation of multi-hop latency. Without an effective optimization strategy, it is difficult for a Service Grid to meet the requirements of low-latency applications.

c. Latency of centralized control: In the traditional service grid architecture, the centralized control plane may lead to a certain delay in the issuance of control commands, especially in large-scale distributed environments, which can significantly affect the real-time response of services.

iii. Challenges of complex traffic management

Complex traffic management requirements stem from a variety of factors, including the diversity of services, the variability of network traffic patterns, and the stringent requirements for quality of service (QoS) and service level agreements (SLAs). Challenges faced by Service Grids in traffic management include:

a. Dynamic Load Balancing: As service instances are dynamically scaled up or down, one of the main challenges faced by service grids is how to effectively load balance to ensure that traffic distribution among service instances is balanced and meets performance requirements.

b. Traffic prioritization and QoS management: Traffic for different services may have different priorities, and the current Service Grid architecture is still deficient in classifying and prioritizing traffic, making it difficult to ensure that traffic for high-priority services receives priority treatment.

c. Abnormal Traffic Detection and Handling: In the face of complex network traffic patterns, Service Grids need to have real-time traffic monitoring and abnormal traffic detection capabilities. However, under large-scale service deployment, real-time monitoring and fast response to anomalous traffic pose performance and scalability challenges to the current service grid architecture.

d. Traffic management across network scenarios: With the development of 5G and edge computing, service grids need to be able to manage traffic across multiple network scenarios (e.g., cloud, edge, IoT). However, current architectural designs mainly target

cloud environments and still lack effective support for traffic management in heterogeneous network environments.

1.2 Research objectives

The research objective of this paper focuses on the design and implementation of an optimized service grid architecture to cope with the high dynamics and complexity of future networks in the future network environment. At the same time, key techniques and methods for intelligent routing, traffic management strategies, and network resilience optimization are explored. Specifically, the following aspects can be developed:

a. Enhancement of Network Dynamic Adaptability: Aiming at the characteristics of high dynamics (e.g., bandwidth fluctuations, delay changes, etc.) in future networks, the research goal is to adjust routing paths in real time through intelligent routing to ensure that network performance metrics such as latency and packet loss can be kept within a controllable range in a complex network environment. Experiments will be conducted to evaluate the adaptability of intelligent routing algorithms to dynamic network conditions by simulating highly dynamic network environments in order to verify their stability and adaptability under network fluctuations.

b. Meeting the demands of low-latency application scenarios: For low-latency application scenarios (e.g., autonomous driving, industrial IoT) in 5G/6G networks, the research goal is to ensure that the Service Grid architecture can achieve stable low-latency transmissions in experiments by optimizing the performance of the Service Grid's data plane, especially the forwarding efficiency of the proxies, as well as the dynamic traffic management techniques in the latency-demanding application scenarios. The experiments examine the reduction of latency after data plane optimization by simulating the load conditions of low-latency applications and compare it with the traditional architecture.

c. Enhanced Load Balancing and Traffic Management: To address the complexity of traffic management in large-scale distributed systems, the research goal is to design and implement an intelligent traffic prediction model, and experimentally test the responsiveness of the traffic management strategy to high-load environments. The experiment will utilize the results of traffic fluctuation prediction to dynamically adjust the load distribution strategy, and verify the system's response speed and resource scheduling capability in the face of unexpected traffic through actual load testing, so as to ensure the system's high availability in traffic fluctuations.

d. Enhancement of system resilience and fault recovery capability: In view of the diversity of future network devices and the high failure rate in distributed systems, the research goal is to realize the fast recovery capability of the system under high load or fault situations by introducing automatic scaling and fault recovery mechanisms. The experiments will validate the auto-scaling and redundant switching capability of the system by simulating service instance failures and resource constraints, and ensure that the recovery time is less than 5 seconds under high load or node failure situations.

2. Related work

2.1 Service Grid Foundation

Service Grids are defined as an infrastructure layer for microservice architectures that help manage communication between microservices, provide security, traffic management, policy and

telemetry, and observability. Service grids simplify the development and management of microservices by abstracting infrastructure-related code away from microservices. Two of the most representative Service Grid solutions are Istio and Linkerd.

1. Istio

   Istio is a service grid jointly developed by Google, IBM, and Lyft, and has become one of the leaders in the service grid space with its rich set of features and extensive community support.Istio's core features include automatic load balancing, fault injection, traffic management, service discovery, and security.

   Istio's core components include the Envoy agent, the Pilot control plane, the Citadel security service, and the Mixer policy and telemetry service.Istio is very powerful in terms of traffic management, distributed tracing, and security (e.g., mTLS encryption), and it supports complex scenarios such as A/B testing and canary publishing [4]. Specifically, Istio offers rich features such as blue-green deployment, circuit breakers, fault injection, and traffic limiting, enabling comprehensive support for large enterprises and complex environments.

   However, this also means that it is more complex to operate and more demanding on the operations and maintenance teams. While Istio has seen performance improvements in recent years, its complex architecture has led to a high resource overhead, especially in terms of control plane scaling.

2. Linkerd

   Linkerd focuses on simplicity and lightweight compared to Istio. v2 of Linkerd focuses on Kubernetes environments and has a simpler architecture with less complexity than Istio's multi-platform support. linkerd is simpler to install and maintain, and because of its lightweight architecture, the data plane scales better than Istio. In addition, Linkerd does not rely on the Envoy agent, but instead uses a micro-agent, which allows it to perform better in resource-constrained environments.

   However, because of its lightweight nature, Linkerd's functionality is relatively limited, especially in terms of traffic management and security management, and it does not have the granularity of control that Istio has. As a result, Linkerd is better suited for small organizations that don't need complex functionality, or for teams that want to reduce O&M overhead.

2.2 Core technologies for service grids and their existing research

Service Grid, as an important part of microservice architecture, realizes the efficient management of inter-service communication through the sidecar proxy deployed next to each service instance. Its core technology covers service discovery, traffic management, fault handling, security, monitoring and tracing, etc. It aims to enhance the observability, reliability and security of microservices in a transparent way, thus allowing developers to focus more on the development of business logic rather than the complexity of the underlying network communication. Common Service Grid implementations such as Istio and Linkerd provide a solid foundation for the architectural design of modern cloud-native applications.

Service grids use mechanisms such as mTLS to secure communication between microservices, with Istio providing comprehensive security features and Linkerd focusing on lightweight and ease of use. However, security isolation in multi-tenant environments, the high maintenance cost of Istio and the security capability limitations of Linkerd, as well as the real-time updating and performance

balancing of security policies in dynamic network environments, constitute the main challenges in the security of service grids. Future research needs to focus on optimizing existing security policies to cope with increasingly complex network environments.

There are also different studies in these areas of load balancing, traffic distribution, Load balancing is a key aspect of cloud computing that avoids the situation where some nodes are overloaded while others are idle or have little work to do. Load balancing improves quality of service (QoS) metrics including response time, cost, throughput, performance and resource utilization.Ghomi et al.

studied the literature on task scheduling and load balancing algorithms and proposed a new categorization of such algorithms [5], such as Hadoop MapReduce load balancing class, agent-based load balancing class, and so on. However most of the existing load balancing methods are handcrafted for the structure of network workloads. Therefore, if the underlying network conditions change, new load balancing strategies are required. On this basis, Chang et al. described a load balancing scheme Learning Load Balancing (LLB) [6], which is a general approach to find the best load balancing policy for a given network topology and workload, fast enough in practice to deploy inferred policies.

Nowadays, computer technology has entered the time of network-centric computing, above the growth of processor speed and memory access speed, more and more bottlenecks will appear on the server side.Zeng et al. analyzed the details of three load balancing techniques and also presented four requirements for the development of the network. The load balancing scheduler design is then carried out accordingly, and then the details of the resulting load balancing are provided for the scheduling system architecture of the group set [7], Other than that, in order to solve the problem that traffic loads in heterogeneous cellular networks (HCNs) are usually unevenly distributed, Liu et al. extended their previous research on traffic load balancing handoff schemes based on the estimation of the user's data rate in realistic HCNs. Numerical results for the balanced scheme show considerable improvement in data rates, especially for low-rate users, compared to the conventional maximum bias received power correlation scheme. The effect of user distribution in hotspots on the balancing results was also discussed [8]. Bhowmik et al. proposed an efficient genetic algorithm based approach for SDN data plane traffic load distribution [9], which effectively reroutes packet flows when such unfavorable situations occur.

In terms of security management, service grids have emerged as attractive DevOps solutions for collecting, managing, and orchestrating microservice deployments [10]. However, current service grids can leave basic security mechanics missing or incomplete. The security burden means that Service Grids may actually create additional workload and overhead for administrators of traditional monolithic systems.Hussain et al. proposed and implemented a security framework [11] for creating secure API Service Grids using Istio and Kubernetes, and, later, proposed an intelligent association model for automatically associating new APIs to existing service grid categories. Although existing APIs are enabled with security measures based on predefined configurations. However, there is no mechanism for dynamic trust evaluation of participating services, which is necessary to protect applications from potential attacks.Sedghpour et al. introduced a runtime trust evaluator (RTE) as a component in the control plane of a service grid [12]. The RTE evaluates the trustworthiness of a service at runtime before establishing service-to-service communication.The core function of the RTE is to evaluate new service invocation requests initiated by a service to assess the trustworthiness of that service.

2.3 Intelligent and Adaptive Technologies for Service Grids

1. **Intelligent routing algorithms:**

With the rapid development of the Internet, users' demand for information sharing has gone beyond the traditional local network and gradually shifted to maximizing the utilization of diverse network resources on a global scale. This trend has significantly boosted the demand for intelligent networks and put forward new challenges and requirements for the evolution of intelligent routing protocols. Modern intelligent routing protocols not only need to have the ability of self-awareness, self-learning, self-decision making and self-configuration, but also must integrate complex functions such as service models, modulation techniques, power control, spectrum awareness and joint optimization to adapt to changing network environments and user demands [13].

Current intelligent routing strategies often rely on static routing decisions when dealing with limited link information due to traffic surges in data center networks. This approach often fails to effectively deal with congestion problems and low bandwidth utilization efficiency caused by complex network environments and dynamic traffic loads. To address this problem, Wang et al. proposed an improved Dueling DQN (Deep Q Network) intelligent routing algorithm [14]. The algorithm incorporates real-time monitoring of network link information by a software-defined network (SDN) controller, which maps this information into an intelligent state space to optimize the routing decision process. In addition, this intelligent routing model employs multidimensional reward weights and facilitates the a priori exploration of Dijkstra's algorithm through the Epsilon greedy strategy. This approach aims to intelligently adjust the routing and forwarding strategies to adapt to the changing network traffic so as to effectively alleviate link congestion and improve the transmission efficiency and overall performance of network traffic. This approach not only enhances the reliability of traffic transmission, but also provides important ideas and references for the further development of intelligent routing protocols.

Currently, reinforcement learning (RL), especially Q-learning, is considered as an effective solution to address the existing challenges in routing policies. It introduces advanced features such as autonomy, adaptivity, and self-learning capabilities to the development of routing strategies.Hosseinzadeh utilized Q-learning to enhance network performance [15] and proposed a Q-learning based routing strategy for Flying Self-Organizing Networks (FANETs), which employs an intelligent filtering algorithm, called QRF, aiming to optimize the network operational efficiency by dynamically adjusting the routing decisions.Zhao et al. proposed a deep reinforcement learning (DRL) multicast routing method based on SDN, DRL-M4MR.In a dynamic SDN environment, DRL-M4MR utilizes a double-duel DQN reinforcement learning approach, which enables the agent to construct multicast trees with the best performance metrics based on the measured network link information after training [16]. In addition, Bai et al. proposed an adaptive intelligent routing algorithm based on the deep reinforcement learning PER-D3QN model [17], which can effectively balance the energy consumption and node load of the network.

Service Grid improves the reliability and elasticity of microservice architectures, but faces challenges in dynamic network environments: its load balancing strategy is inefficient in dealing with bursty traffic; in complex network topologies (e.g., cross-geographic data

centers), the routing strategy is difficult to deal with network delays efficiently, which may lead to misrouting and increase in response time; and the communication delays between the control plane and the data plane affect the traffic management strategy of the real-time updates.

Istio and Linkerd still face challenges in fine-grained security segregation in multi-tenant environments: Istio's complexity leads to high maintenance costs, while Linkerd's functionality may be insufficient. Balancing real-time updates of security policies with performance in dynamic environments is also a key issue. Therefore, the traffic management and security policies of existing service grids need to be further optimized for future complex network environments.

2. **Dynamic Load Balancing and Traffic Prediction:**

In modern microservice architectures, service grids significantly enhance system efficiency and responsiveness through dynamic load balancing and traffic prediction techniques. These techniques give Service Grids the ability to intelligently allocate resources under fluctuating traffic environments, thus improving system stability and performance. Dynamic load balancing, as the core mechanism for real-time traffic allocation, can monitor the service instance status and system load in real-time, dynamically adjust traffic allocation, effectively avoid single instance overload, and achieve optimal utilization of system resources, compared with traditional polling or minimum number of connections strategies.

In the research field of dynamic load balancing algorithms, Mochalov et al. proposed a dynamic load allocation and balancing algorithm for distributed cloud computing environments [18], which provides an effective solution for the reasonable allocation of cross-server virtual machines. While Zhong et al. proposed a prediction-based dual-weight switch migration scheme for SDN load balancing for multiple distributed controllers [19]. This scheme achieves fast load balancing among controllers and reduces the number of switch migrations by using historical traffic load data for predicting future traffic load.

Accurate traffic prediction is critical for optimizing service grid performance. Prediction based on historical traffic data and machine learning models enables pre-provisioning of resources and dynamic adjustment of load balancing to effectively respond to traffic bursts, reduce response latency, avoid service disruptions and improve resource utilization. This approach has significant implications for cost-effective network management, resource allocation and security policies, especially in software-defined networks and zero-trust network environments.

Kablaoui proposed an integrated Machine Learning (ML) model [20] that combines pre-trained AlexNet and ResNet models, 2D Convolutional Neural Networks (CNNs), Visual Transformers (ViTs) as well as LSTM/GRU models and their variants for network traffic prediction. To configure the network more efficiently, Li et al. proposed two new mobile traffic predictors [21], the first one is the Mixed of Experts (MoE) model, which significantly improves the accuracy and interpretability of peak prediction by constructing a collaborative mechanism between different experts, and the second one is the Lightweight Multi-Layer Perceptron (MLP), which obtains a similar performance of peak prediction while being more flexible and less computationally

intensive. The prediction results are used to assist in predictive detection of traffic imbalance.Wang et al. introduced a hybrid deep learning approach equipped with an attention mechanism [22] to propose a novel traffic prediction algorithm for mobile networks, which analyzes the traffic in the cellular area and determines the spatio-temporal characteristics of the residential community to achieve efficient mobile traffic prediction.

Although dynamic load balancing and traffic prediction techniques have made significant progress in improving the efficiency of service grids, they still face many challenges in practical applications. First, dynamic load balancing relies on accurate monitoring and fast response to real-time data; however, in highly distributed systems, delays in data synchronization and processing may reduce the effectiveness of load balancing. Second, traffic prediction models are difficult to cope with the diversity of traffic patterns and sudden changes in complex network environments. Finally, how to reduce computational overhead while ensuring prediction accuracy is an important direction for future research.

3. **Network Resilience and Failure Recovery:**

Cyber resilience, which measures the ability of a complex system to adjust its activities in order to preserve the essential functionality of the system from errors or failures, is of increasing interest in various fields. Resilience analysis plays an important role in early warning, predicting and proposing potential strategies or designing optimally resilient systems [23]. Network resilience and failure recovery ensure high availability and stability of service grid systems in the face of various uncertainties and contingencies. Two of the most common ones are Auto-scaling and redundant path management.

a. Auto-scaling: Auto-scaling allows the system to automatically adjust the number of service instances based on the current load conditions to cope with load fluctuations. This is usually achieved by combining Kubernetes' Horizontal Pod Autoscaler (HPA), which dynamically scales up or down service instances based on metrics such as CPU and memory usage to ensure efficient utilization of system resources and stable performance.
b. Redundant Path Management: Redundant path management improves system fault tolerance by configuring multiple service paths or redundant service instances. In a service grid, this mechanism ensures that when a path or service instance fails, traffic can be quickly switched to other available paths or instances, avoiding service disruption caused by a single point of failure.

These techniques are effective in highly dynamic network environments such as edge computing. Auto-scaling technology dynamically adjusts the number of edge node service instances based on real-time traffic and load changes to ensure service stability, while redundant path management technology ensures service continuity and reliability in the event of node failure through alternate paths.

Although auto-scaling and redundant path management enhance service grid resilience and fault recovery, they still have limitations in high-frequency failures and complex network environments: frequent instance startup/shutdown may cause resource bottlenecks; path switching may lead to latency and resource overhead; coordination

between the control plane and the data plane limits recovery speed in highly distributed environments, and the response may lag behind, especially in the case of large-scale failures. .

2.4 Research on Service Grid Optimization for Future Networks

In future network environments (e.g., 5G/6G, IoT, Edge Computing), Service Grids, as an important part of microservice architectures, need to face the challenges of high dynamics, low latency requirements, and complex traffic management.

1. **Research on Adaptation to Future Networks**

a. Characterization of future networks:

i. High Speed: Supports higher data transfer rates to meet the needs of high volume data.

ii. Low latency: realizing real-time communication and providing immediate response for various applications.

iii. Intelligence: self-optimization and self-management using artificial intelligence and big data analysis.

iv. Reliability: Enhance the stability and anti-attack capability of the network to ensure the continuity of service.

v. Interconnectivity: Supporting the collaborative work of multiple devices and networks to promote the interconnection of everything.

vi. Green: Focus on energy efficiency, reduce carbon footprint and promote sustainable development.

vii. Security: Emphasize data privacy and security, and provide multi-layered security mechanisms.

b. Existing research

5G architectures mark a significant advancement in network technology. However, with the dramatic increase in the volume of messages in 5G services, the complexity of the infrastructure increases significantly, which poses serious challenges for request processing and system resilience.Aldas et al. investigated the complexities faced by 5G in modern cloud-native deployment environments, with a particular focus on inter-service communication, service discovery, and network programmability. They highlighted the key role of Service Mesh in realizing the 3GPP vision and proposed customized enhancements for 5G deployment scenarios [24]. In addition, this research provides important architectural guidance for optimizing the Service Mesh framework to support 5G core signaling.Duong et al. proposed a Service Mesh-based 5G core network architecture leveraging the Cilium Container Network Interface (CNI) plug-in [25] and demonstrated the potential of Service Mesh technology in enhancing the security and observability of 5G core networks (5GC).

As the market demand continues to grow and its expansion into multiple industries, the design of 5G next-generation cores is trending towards service-based architectures.Ungureanu et al. proposed a novel approach to deploy 5G next-generation core functionality as a cloud-native application [26], which is capable of flexibly scaling up as per the requirements to accommodate constraints in different verticals. In the transition from 5G to 6G, the focus has shifted to enabling flexibility, scalability, and automation of mobile core networks in a multi-cloud environment. Despite significant

progress in 5G, its full realization is still in the emerging phase.Thai et al. devised an innovative core network architecture [27] that transforms network functions into cloud-native applications by separating the service layer from the deployment and operations layer and introducing a native service grid. In this context, post-fifth-generation mobile (B5G) has received much attention and is being actively developed in various countries.SAITO et al. proposed an information-centric service mesh (IC-SM) [28], which integrates cloud-native service grids through name-based dynamic routing, enabling devices to "autonomously" collaborate without human intervention. "Autonomous" Collaboration.

The increasing deployment of Mobile Edge Computing enhances the capabilities of 5G infrastructures to further enable smart data processing in smart city scenarios. Future innovative city applications require seamless integration of mobile edge computing and communications.Yang et al. provide a comprehensive overview of next-generation communication networks for integration with distributed computing in academia and industry [29], supporting Networks for Artificial Intelligence (NET4AI) and Edge Grid Computing architectures that enable cloud-native functionality to be extended to the communication network for integrated communication and computing services.

c. Limitation

Despite the progress of existing research in terms of adaptability, it still faces many limitations in future network environments. First, although edge computing can effectively reduce latency, the response time of service grids in very low-latency application scenarios (e.g., telemedicine and autonomous driving) still needs to be further optimized. Second, the highly dynamic nature of the future network environment puts higher requirements on the ability of the service grid to perceive the network state in real time, while the existing monitoring and adjustment mechanisms may have a lag in the complex network environment, making it difficult to respond to unexpected changes in a timely manner. These problems need to be studied and solved in depth to improve the adaptive performance in the future network environment.

2. **Cross-domain and multi-protocol support**

a. Existing research

Service Grids are being widely adopted as a cloud-native mechanism to enable traffic management in applications based on microservices architectures, especially in general-purpose IT workloads in increasingly centralized cloud environments. However, for performance-critical applications, the trend towards decentralization of modern application execution environments, especially in mobile edge cloud scenarios, continues to drive the development of Service Grids.Duque et al. conducted a systematic and qualitative analysis of current state-of-the-art Service Grids [30] to assess whether their design can effectively meet the high-performance application workloads deployed in mobile edge cloud environments traffic management needs of loads deployed in mobile edge cloud environments. It was found that the reliability-centered service grid design fails to adequately meet the diverse requirements of emerging mobile edge cloud workloads. Therefore, future research should focus on developing architectures with higher performance efficiencies, enhanced quality of

service (QoS) assurance mechanisms, and more sophisticated cloud-native traffic management frameworks to adapt to the evolving technology landscape.

Elkhatib et al. compared the performance and overhead of the two most popular service grids (Istio and Linkerd) on resource constrained devices [31]. Through extensive experiments and comparison with a baseline of standard networks in Kubernetes clusters, it was found that Linkerd provides a friendlier service grid option in edge computing environments compared to Istio. as an alternative or complement to cloud computing, edge computing has the potential to extend microservices to edge environments, thus simplifying the development and deployment of edge applications. li et al. proposed an edge service grid architecture [32], which includes distributed controllers for edge-native computing.

The Internet of Things (IoT) is rapidly emerging, and according to recent data, the number of connected IoT devices (both cellular and non-cellular) will reach 25.1 billion globally by 2025. To effectively support this growth trend, it is critical to build IoT edge applications using cloud-native technologies based on microservices, containers, and service grids. Cloud-native technologies give these applications significant scalability, rapid deployment capabilities, and flexibility to adapt, with the core component being modularizable containers that communicate across the network via REST or gRPC interfaces.

Cloud-native architectures place higher demands on data transfer and control information exchange than traditional monolithic application architectures, and therefore require more robust data throughput capabilities. In this context, the Shimmy model emerged [33], a novel memory-based shared communication interface designed to achieve efficient integration with containers in container orchestration systems such as Kubernetes. Experiments have demonstrated that Shimmy is able to significantly reduce latency when handling large data transfers and real-time video streaming [34]. Compared with traditional network protocols (e.g., TCP/UDP, which increase latency by a factor of 3 to 4) and message queues (e.g., RabbitMQ, which increase latency by a factor of 20), Shimmy's performance performance is particularly impressive, while helping to reduce memory and CPU utilization.

At the same time, the evolution of network technologies has given rise to a variety of new network protocols that are significantly different from the traditional TCP/IP protocol. However, existing protocol conversion mechanisms usually support only a few predefined protocols and lack the flexibility to scale to emerging protocols, which is not sufficient for the efficient convergence of heterogeneous networks. Furthermore, in core networks, data is at risk of theft and tampering, especially in the absence of effective security measures. To address this challenge, Liu et al. proposed MultiSec [35], a multi-protocol secure forwarding mechanism based on programmable data planes, which aims to provide cryptographic protection for heterogeneous protocols transmitted through core networks.MultiSec leverages the flexibility and definability of programmable network architectures to support flexible protocol extensions, thereby facilitating the convergence of various heterogeneous networks and improving forwarding efficiency and data security.

b. Limitation

Cross-domain service collaboration and multi-protocol support in future networks face a number of challenges, among which network latency and data consistency issues are particularly prominent. First, due to the differences in physical distance and network conditions between different network domains, network latency in cross-domain scenarios may significantly affect the performance of service grids. Second, in the dynamically changing network environment, the data consistency problem brought about by cross-domain collaboration is still a difficult problem, and Service Grid needs to effectively deal with data inconsistencies caused by data transmission and processing delays in different domains, in order to ensure the reliability and stability of the system as a whole.

3. Architecture Design

3.1 Overall Architecture Design

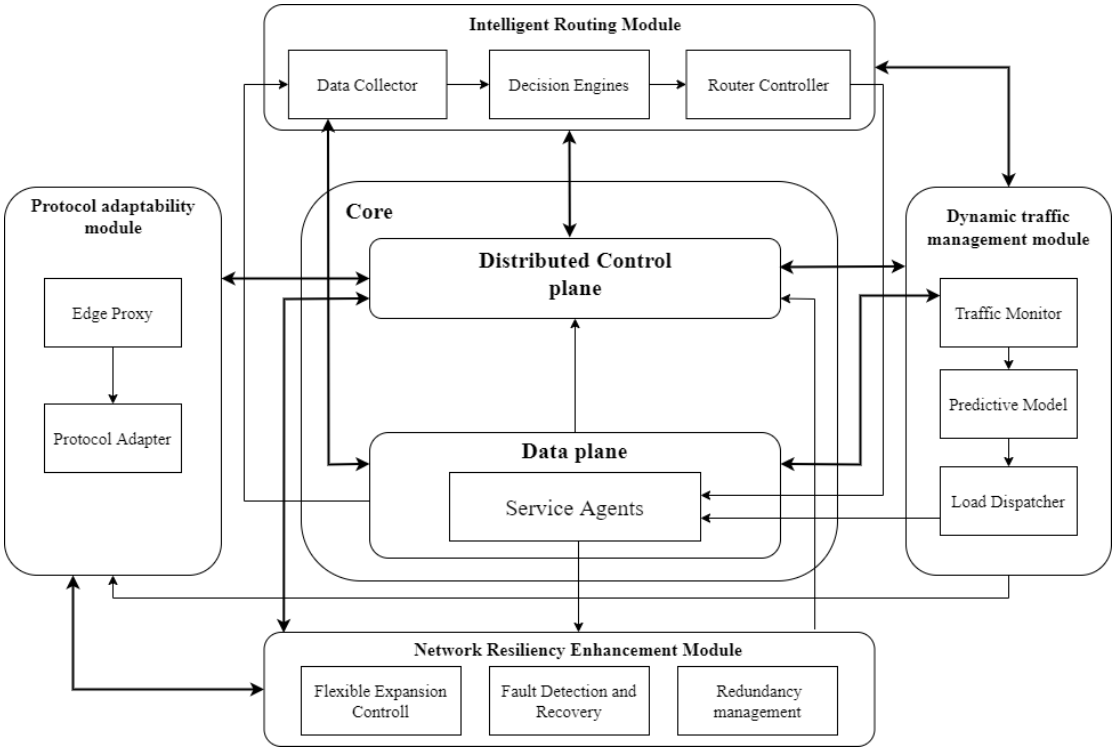


Fig. 1 Overall Architecture Diagram

As shown in Fig. 1, the overall architecture of the Service Grid is designed to cope with the dynamic complexity of future network environments, and to realize intelligent network management through the collaboration of distributed control plane and data plane. The Intelligent Routing module utilizes real-time data collection, decision engines and routing controllers to achieve dynamic optimization of network paths and ensure efficient data transmission. The Dynamic Traffic Management module dynamically adjusts the allocation of network resources through traffic monitoring and prediction models, combined with a load scheduler, thus improving network adaptivity and service quality.

In addition, the Protocol Adaptation module enhances the compatibility of multi-protocol networks through edge agents and protocol adapters, enabling different protocols to collaborate efficiently in a unified control and data plane. The Network Resiliency Enhancement Module provides flexible expansion control, fault detection and recovery mechanisms, and redundancy

management policies to further enhance the network's disaster-tolerance capability and stability, thus ensuring continuous and reliable services in various network environments. Through the integration of the above modules, the architecture achieves intelligent, adaptive and efficient optimization of the service grid in future network environments. In order to have a clearer understanding of the overall operation process of the architecture, Figure 2 shows the workflow timing diagram of the architecture.

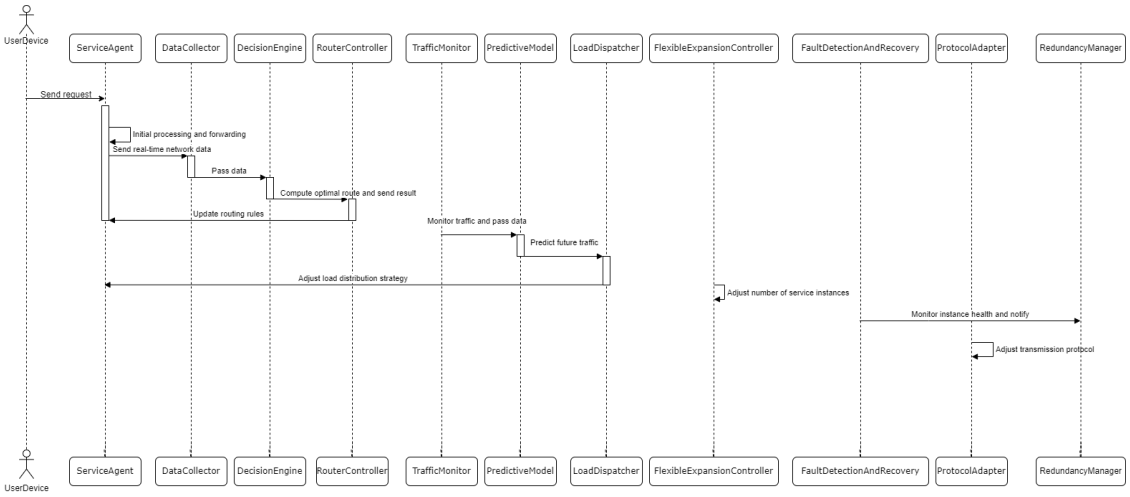


Fig. 2 Timing diagram of the workflow of the architecture

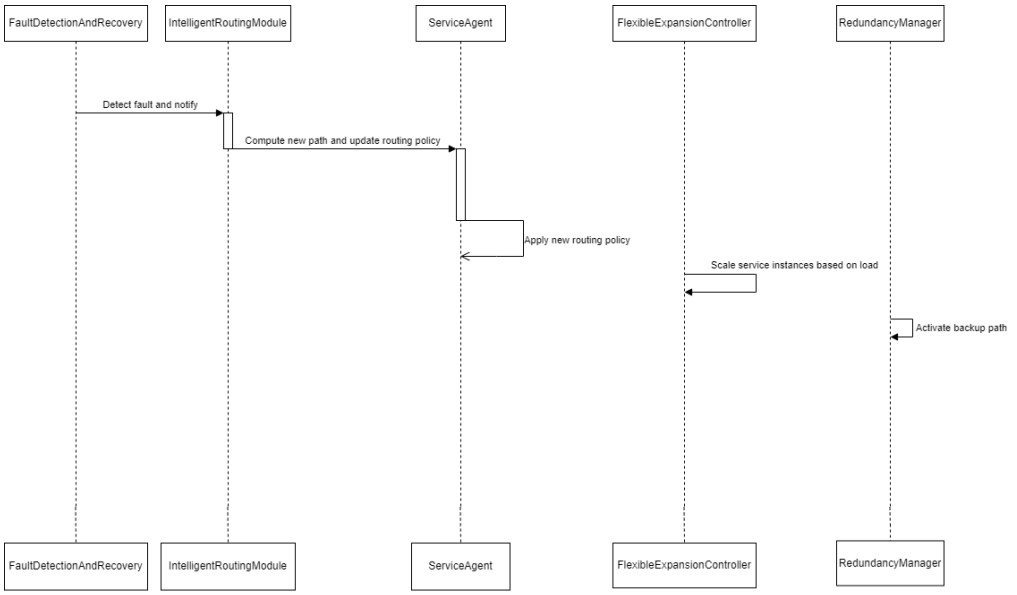


Fig. 3 Fault detection and recovery

The key interaction steps of the fault detection and recovery process in the architecture are illustrated in Fig. 3, which demonstrates the system's adaptive response mechanism in the face of network anomalies. First, the fault detection and recovery module discovers and recognizes faults in real time by continuously monitoring the network status, and subsequently notifies the intelligent routing module to respond. Upon receiving the fault notification, the intelligent routing module quickly calculates and updates a new routing policy based on real-time network conditions and preset decision-making algorithms to ensure that the data flow can continue to be transmitted efficiently on the controlled path.

During the implementation of the new policy, the service agent module is responsible for applying the updated routing configuration, while the flexible scaling controller dynamically adjusts the expansion and contraction of service instances based on the current network load, optimizing the resource allocation to adapt to the changing demands. In addition, the redundancy manager activates the backup path immediately after detecting the failure of the critical path, which further enhances the disaster recovery capability of the system and ensures the continuity and stability of the service. Through the close collaboration between this series of modules, the system achieves rapid response to network anomalies, path optimization, and dynamic scheduling of resources, building an efficient, reliable, and adaptive future network architecture.

3.2 Intelligent Routing Module Based on Deep Q Learning

In future network environments, network conditions will change at any time, such as latency, bandwidth, load, and so on. The task of the intelligent routing module is to dynamically select the optimal path based on the real-time network state in order to reduce latency and improve the quality of service.

A. Algorithmic mathematical model

a. State and Action Definitions

Definition of state space S and action space A:

- State $s \in S$ denotes the current state of the network, e.g., network delay, bandwidth, packet loss rate, etc.
- The action $a \in A$ denotes the routing path or policy chosen by the agent.

b. Reward function

The design of the reward function $R(s,a)$ takes into account the penalties for delay, packet loss rate, and path switching:

$$R(s,a) = -Latency(s,a) - \alpha \times PacketLoss(s,a) - \beta \times PathSwitchingPenalty(s,a)$$

Among them:

- $Latency(s,a)$ is the network latency to take action a in state s.
- $PacketLoss(s,a)$ is the packet loss rate of taking action a in state s.
- $PathSwitchingPenalty(s,a)$ is the path switching penalty, calculated as:

$$PathSwitchingPenalty(s,a) = \gamma \times | a - PrevAction(s) |$$

α, β and γ are the weighting factors of the reward function.

c. Q-value update

The Q-values are updated using an improved target network mechanism and an empirical playback mechanism:

$$Q(s,a) \leftarrow (1-\lambda) \times Q(s,a) + \lambda \times [R(s,a) + \delta \times \max_{a'} Q(s',a')]$$

Among them:

- λ is the learning rate.
- δ is a discount factor that weighs the importance of future rewards.
- $Target\ Network$ is used to minimize the bias in the estimation of the Q-value and is updated every N steps.

d. Updates to the target network

Every T steps, the weights of the target network are updated to the weights of the current model:

$$TargetNetwork \leftarrow CurrentModel$$

e. Adaptive exploration strategies

Claim rate ϵ Changes over time, higher initially to encourage exploration, gradually decreasing later to encourage utilization:

$$\epsilon_t = \epsilon_{min} + (\epsilon_{max} - \epsilon_{min}) \times e^{-\lambda_t}$$

Among them:

- ϵ_{min} is the minimum exploration rate.
- ϵ_{max} is the initial exploration rate.
- λ_t is the attenuation factor associated with the time step t.

f. Replay of experience

An empirical playback mechanism is introduced to improve sample efficiency and training stability:

$$ExperienceReplay(s, a, r, s')$$

During the training process, empirical batches are randomly selected for training to break the temporal correlation between samples.

g. Dynamic incentive adjustment

The dynamic adjustment mechanism in the reward function is as follows:

$$R(s, a) = -Latency(s, a) - \alpha \times PacketLoss(s, a) - \beta \times (\gamma \times \mid a - PrevAction(s) \mid)$$

B. Description of algorithms

In the implementation of the intelligent routing module, a Deep Q Learning (DQN) algorithm is used to cope with dynamic changes in network conditions. The module optimizes the path selection strategy by simulating the changes in network latency and packet loss rate, using the Q-learning approach in reinforcement learning, and by approximating the Q-function through neural networks. During the training process, the module adjusts the path selection by dynamically exploring and exploiting the strategies, and introduces a path switching penalty mechanism to balance the network performance and path stability. Eventually, the trained DQN model is able to make optimization decisions in real-time network state to improve the quality of service and stability of the network. The innovation of the algorithm is reflected in the introduction of reinforcement learning techniques such as adaptive exploration strategy, path switching penalty mechanism, experience playback and target network update, and the design of a reward mechanism for network routing optimization in combination with the dynamic changes of the simulated network environment. Through these innovations, the algorithm is able to effectively balance exploration and utilization under variable network conditions, reduce path switching frequency, and improve system stability, thus achieving better path selection and service quality improvement.

The network state simulation section provides a near-realistic training environment for the intelligent routing algorithm by simulating the dynamic changes of delay and packet loss rate. This section designs a mechanism in which the base delay and packet loss rate are gradually reduced with the number of training iterations to simulate the process of network performance enhancement, and introduces normally distributed random noise to reflect the short-term fluctuations and unpredictable factors in real networks. The gradual reduction of the base delay from 45 ms simulates the scenario of network congestion relief and performance optimization, while the addition of random noise helps the algorithm to adapt to unexpected network conditions. Similarly, the initial packet loss rate starts at 50% and gradually decreases with the number of iterations, simulating a gradual improvement in network stability. In the process, random fluctuations simulate episodic problems in the network, making the training closer to the real scenario. The advantage of this simulation method is that it allows the algorithm to make optimization decisions under various network conditions, improves the robustness and adaptability of the algorithm, and ensures its effectiveness and stability in practical applications. Through this detailed simulation, intelligent routing algorithms are able to be effectively trained in complex and dynamic network environments, resulting in lower latency and packet loss rates and optimized path selection strategies. In order to visualize the workflow of the intelligent routing module more clearly, the following is the workflow of the algorithm, as shown in Fig. 4：

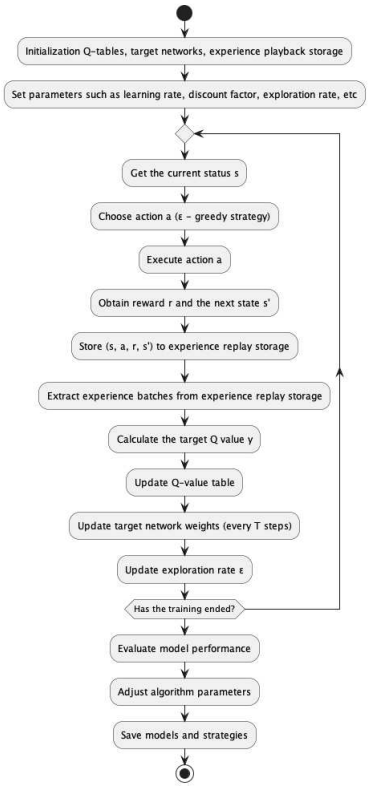


Fig. 4:Flowchart of intelligent routing algorithm.

To elucidate the role of the intelligent routing module in the system architecture, as shown in Fig. 5, which shows the work that the intelligent routing is responsible for when working on the architecture, the following Fig. 5 shows the timing diagram of the work of the intelligent routing module, firstly, the data collection module goes to obtain the network status and performance metrics. Next, this data is passed to the decision engine in the intelligent routing module, where

learning algorithms are utilized for analysis and inference to select the optimal routing path. After outputting the selected routing path, it is passed to the routing module. The routing module updates the routing configuration of the Istio based on the suggested path and applies the new configuration to the data plane of the Istio to optimize the network transmission path of the service。

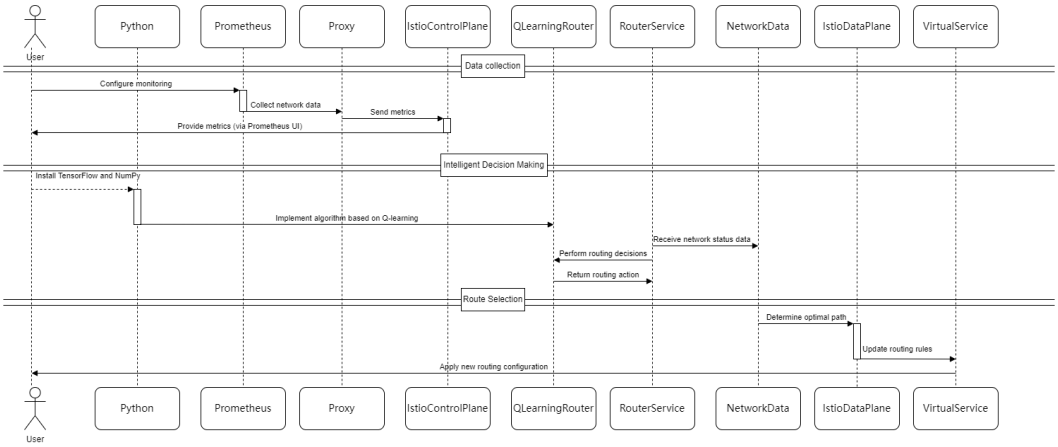


Fig. 5:Intelligent Routing Module Working Timing Diagram.

3.3 Dynamic Traffic Management Module Based on CNN and LSTM

In the face of large-scale traffic fluctuations, the system needs to adjust the load balancing strategy in real time to ensure the optimal utilization of resources and high availability of services. As Fig. 6 shows the detailed working timing diagram of traffic prediction in the whole architecture:

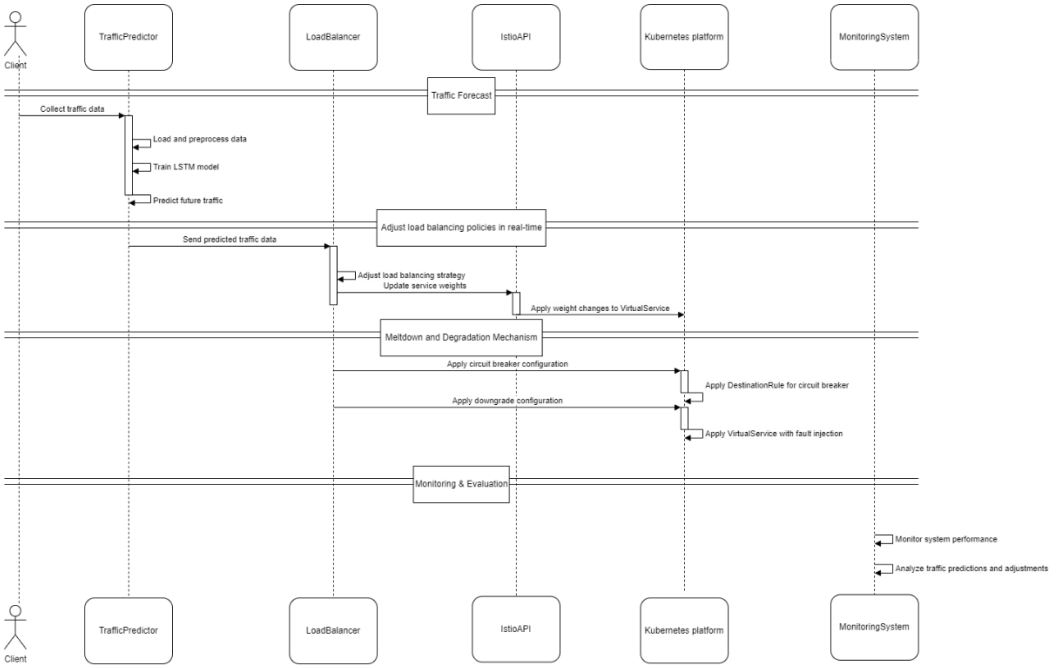


Fig. 6:Timing diagram of flow prediction work.

A. Algorithmic implementation

This algorithm employs a hybrid modeling architecture combining Convolutional Neural Networks (CNNs) and Long Short-Term Memory Networks (LSTMs) for processing complex patterns and dependencies in time series data. Its working principle can be described in detail in several steps such as data preprocessing, feature extraction, time dependency modeling, and prediction generation.

a. Data preprocessing

First, the original time series data are normalized to map the data linearly into the range of [0,1]. This step accelerates the training process of the model and prevents the problem of vanishing or exploding gradients caused by too large differences in the eigenvalues. In addition, the time series data are converted into the form of supervised learning, i.e., the construction of the feature matrix and the target vector. In the construction process, each input sample consists of historical data in a fixed time window, and its corresponding target is a future value after the time window. This data structure is able to capture the dynamically changing characteristics of the time series.

b. Convolutional feature extraction

In the feature extraction phase, the algorithm utilizes a one-dimensional convolutional neural network (Conv1D) to perform local feature extraction on the input time series. The convolution operation extracts patterns or features localized in the time series by performing local convolution operations on the time axis through sliding filters. Multiple convolution filters can capture different scales or different types of patterns to generate a multi-channel feature map. In order to reduce the dimensionality and complexity of the feature maps, a MaxPooling1D layer is introduced immediately after, which retains important features by taking the maximum value within the pooling window, reducing the amount of data and improving the robustness of the model.

c. Time-dependent modeling

The feature maps output from the convolutional layer contain local information about the time series, but lack the ability to model long-term dependencies. In order to capture the long-term dependencies in the time series, the algorithm further inputs the feature maps output from the convolutional layer into the Long Short-Term Memory (LSTM) network.The LSTM controls the flow of information through its unique gating mechanism (input gate, forget gate, output gate), which effectively solves the problems of gradient vanishing and gradient explosion in the traditional RNN. As a result, the LSTM layer is able to capture long-term dependencies in the time series and generate information that represents the global dynamics of the time series.

d. Predictive generation

After processing through the LSTM layer, the higher-order features of the time series are mapped into a low-dimensional vector that carries the global information of the time series. Subsequently, this low-dimensional vector is linearly transformed through a fully connected layer (Dense Layer) to generate the final prediction values. The weight matrix of the Dense Layer maps the output of the LSTM layer to the target space, forming the final output of the model.

e. Model training and optimization

During training, the algorithm optimizes the parameters of the model by minimizing the mean square error (MSE).The MSE measures the gap between the model's predicted and actual values, and the optimization goal is to minimize this gap. The model performs gradient descent using the Adam optimizer and adjusts the weights of the CNN and LSTM layers through a back propagation algorithm to gradually improve the model's prediction accuracy.

f. Prediction and inverse normalization

In the prediction phase, the trained model predicts new time series data and generates output values that are normalized predictions. To obtain the actual flow values, the algorithm back-normalizes the predictions, i.e., restores them to the scale of the original data. Eventually, the model outputs the predicted values of the time series through these steps and the performance of the model is evaluated by calculating the MSE.

B. Algorithmic mathematical models

In network traffic prediction, our goal is to predict future traffic values based on past traffic data. Assume that the sequence of traffic data is $x(t)$, where t is the time index and $x(t)$ is the value of network traffic at moment t. The mathematical structure of the entire model is described in detail in the following steps.

a) Data pre-processing

- Normalization: In order to make the model converge faster and avoid the data scale affecting the model performance, the flow data is first normalized by scaling the data $x(t)$ to the range of $[0,1]$.

$$x_{norm}(t) = \frac{x(t) - min\ (x)}{max\ (x) - min\ (x)}$$

Here, $min(x)$ and $max(x)$ are the minimum and maximum values of the flow data, respectively.

- Time series data construction: the historical flow data is sliced by time step, given the time step $time_step$, the data X is constructed as follows:

$$X(i) = [x_{norm}(i), x_{norm}(i+1), \dots, x_{norm}(i + time_step - 1)]$$

The corresponding target output is:

$$y(i) = x_{norm}(i + time_step)$$

b) Convolutional layer (Conv1D)

Convolutional operations: the purpose of convolutional layers is to extract localized features in a time series. In network traffic prediction, the convolutional layer helps to identify short-term fluctuations or periodic patterns in the traffic data that may reflect sudden increases, decreases, or regular changes in traffic over a short period of time. For a given input sequence $X(i)$, the operation of the convolutional layer is to perform convolution using a filter w_k with $kernel_size$.

$$z_k(t) = \sum_{j=0}^{m-1} w_k(j) \cdot X(i + t + j)$$

where t is the time step, k denotes the kth filter, and m is the size of the convolution kernel. The output of the convolutional layer is the feature map $Z(t)$:

$$Z(t) = [z_1(t), z_2(t), \dots, z_K(t)]$$

Here K is the number of filters.

c) Pooling layer (MaxPooling1D)

Pooling operation: the pooling layer is used to reduce the size of the convolution output, retaining important information and reducing computational complexity. For traffic data, the pooling layer can filter out the short-term high-frequency noise and retain the main trends in the traffic data, helping the model to focus more on the overall traffic trends. A commonly used method is MaxPooling, which takes the maximum value within the pooling window, which is $pool_size$:

$$z_k'(t) = max\{z_k(t), z_k(t+1), \ldots, z_k(t+pool_size-1)\}$$

d) Long and short term memory network (LSTM) layer

- Input gate $\boldsymbol{i_t}$: determines to what extent the current input affects the memory cell (cell state).

$$i_t = \sigma(W_i \cdot [h_{t-1}, z_t'] + b_i)$$

- Forgetting gate $\boldsymbol{f_t}$: controls how much of the previous moment's memory state is retained into the current state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, z_t'] + b_f)$$

- Candidate cell state $\boldsymbol{\tilde{C}_t}$: generates a candidate new cell state based on the current input and the previous moment's hidden state.

$$\tilde{C}_t = tanh\,(W_c \cdot [h_{t-1}, z_t'] + b_C)$$

- Cell state $\boldsymbol{C_t}$: combines forgetting gates and input gates to update the current cell state and capture long time dependencies.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

- Output gate o_tControls how many cell states are output:

$$o_t = \sigma(W_o \cdot [h_{t-1}, z_t'] + b_o)$$

- The final output h_t:

$$h_t = o_t \cdot tanh\,(C_t)$$

Here, σ is the $sigmoid$ activation function, $tanh$ is the hyperbolic tangent activation function, W_i, W_f, W_C, W_o are the weight matrices, and b_i, b_f, b_C, b_o are the bias terms.The LSTM layer is particularly suitable for dealing with long- and short-term dependencies in time series data. In traffic forecasting, LSTM can effectively capture long time trends in historical traffic (e.g., day-night variation, weekday vs. weekend traffic variation) while retaining sensitivity to sudden traffic changes.

e) Full connectivity layer (Dense)

a. Linear transformation: the output of the LSTM is passed through a fully connected layer (linear layer) to generate the final predicted values.

$$\hat{y}_t = W \cdot h_t + b$$

b. Application to flow data: the fully connected layer integrates the features from the LSTM output and outputs a single value as the predicted future flow value.

Here, W is the weight matrix of the fully connected layer and b is the bias. The flow of the algorithm is shown in Fig. 7:

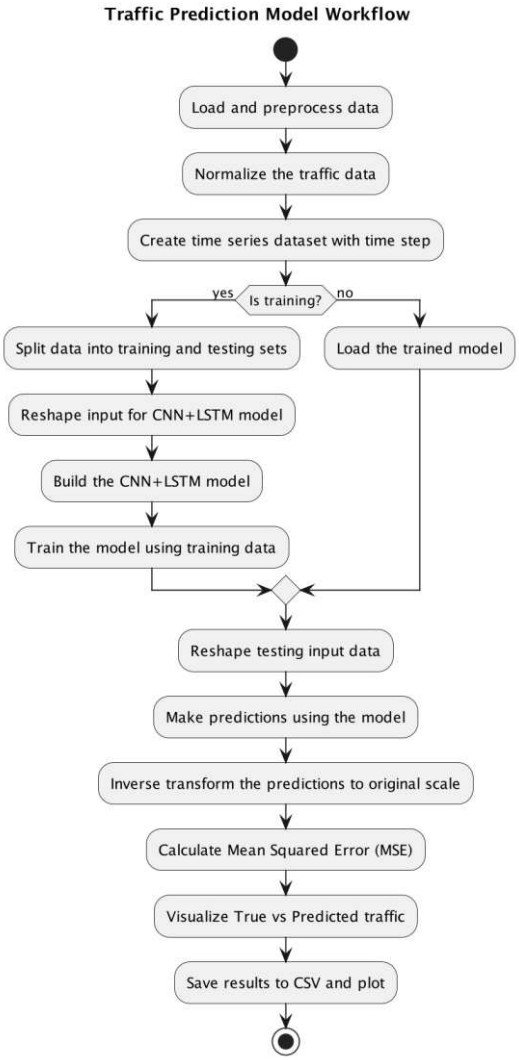


Fig. 7:Flow of traffic prediction algorithm.

C. Forecasting and post-processing

a. Loss Function: Mean Square Error (MSE) is used to measure the difference between the model's predictions and actual flows. The mean square error is defined as:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

b. Optimization process: the model parameters (weights and biases) are continuously updated to minimize the loss function through backpropagation and gradient descent (using the Adam optimizer).

c. Forward propagation: In the prediction phase, the model receives new traffic data and passes it through convolutional, pooling, LSTM and fully connected layers to generate traffic predictions for future moments.

d. Back-normalization: back-normalization of forecasts from the normalized range to the original data range for comparison with actual flow data.

$$\hat{y}_{original}(t) = \hat{y}_t \times (max(x) - min(x)) + min(x)$$

e. Evaluation of results: The predictive accuracy of the model is evaluated using the mean square error (MSE), and the actual versus predicted flow curves are visualized graphically.

D. Load Balancing Policy Adjustment: design the algorithm to change the load balancing policy by updating the weights according to the size of the predicted traffic forecast. The flowchart of the algorithm is shown in Fig. 8:

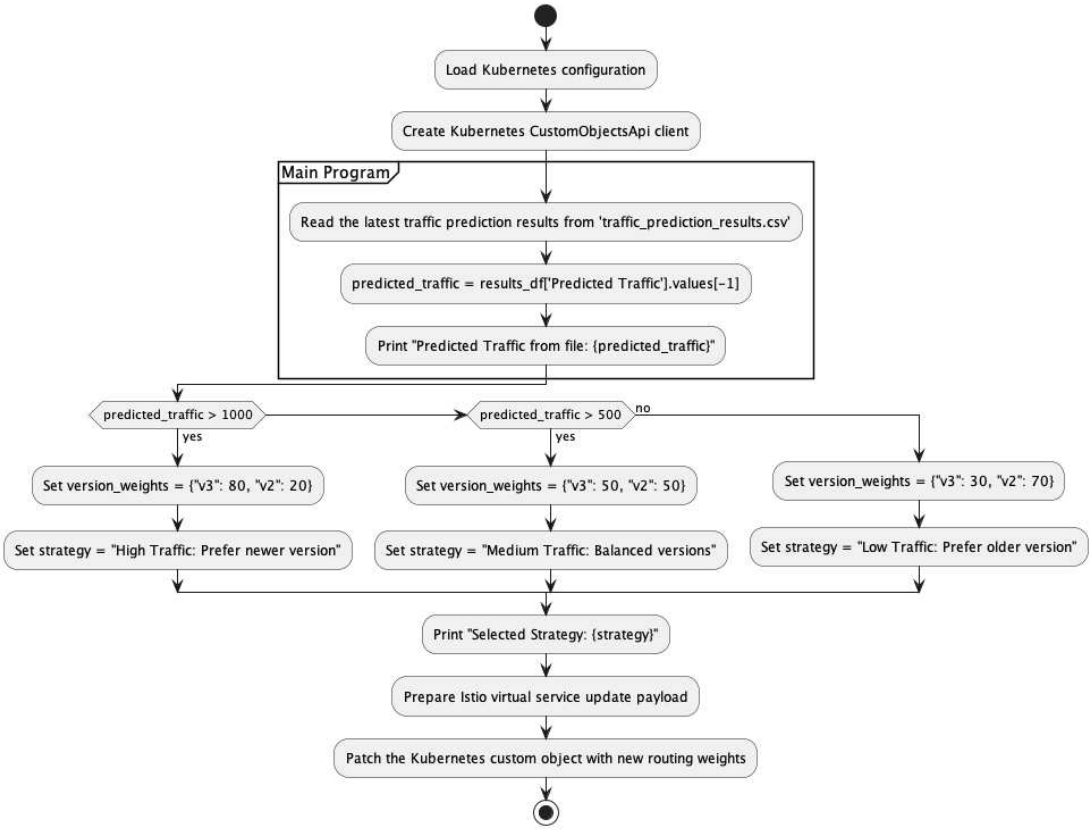


Fig. 8:Load balancing algorithm.

E. Fusing and Degradation Mechanism: When the traffic is too large or the network is abnormal, the fusing or degradation policy is automatically triggered to ensure the stability of the system, as shown in Table 1.

Table 1: Configuration of fusion and degradation mechanisms.

Configuration 1 circuit breaker
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
name: reviews-circuit-breaker
spec:
host: reviews
trafficPolicy:
outlierDetection:
consecutiveErrors: 7

```
    interval: 1m
    baseEjectionTime: 3m
    maxEjectionPercent: 100
```

3.4 Design and Implementation of Network Resilience Enhancement Module

a. **Elastic scaling realization**

Elastic scaling is designed to respond to dynamic changes in service traffic by automatically adjusting the number of microservice instances to maintain a balance between performance and resource utilization. This mechanism realizes the ability to automatically scale up or down compute resources based on demand by dynamically scheduling resources. Instead of designing new algorithmic configurations for elastic scaling, this section utilizes the horizontal auto-scaling policy (HPA) that comes with Kubernetes itself, as shown in the following table. This policy adapts to different load scenarios by increasing or decreasing the number of Pods (containers).The Horizontal Pod Autoscaler (HPA) in Kubernetes is the core component that enables horizontal scaling, which dynamically adapts the number of Pods based on the resource usage of each Pod in the cluster (e.g., CPU, memory, etc.), as shown in Table 2.

Table 2:Horizontal Auto Scaling Policy Configuration.

Configuration 2 hpa

```
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: productpage
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: productpage-v1
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 80
```

b. **Fault automatic recovery**

The goal of automatic fault recovery is to ensure high system availability by quickly switching to a standby instance or triggering a disaster recovery strategy when a service

instance failure is detected. the automatic fault recovery strategy is shown in the following table. This mechanism realizes real-time monitoring and recovery of service instances through health checking and automatic failover.The health checking mechanism in Kubernetes decides whether to restart or quarantine a failed instance by periodically checking the health status of the service instance.The Liveness Probe is used to determine whether the instance is alive or not, and the Readiness Probe is used to determine whether the instance is ready to handle traffic, as shown in Table 3.

Table 3:Automatic Fault Recovery.

Configuration 3 Automatic fault recovery
apiVersion: autoscaling/v2beta2 kind: HorizontalPodAutoscaler metadata: name: productpage spec: scaleTargetRef: apiVersion: apps/v1   kind: Deployment   name: productpage-v1   minReplicas: 1   maxReplicas: 10   metrics: - type: Resource   resource: name: cpu   target: type: Utilization   averageUtilization: 80

LivenessProbe ensures that the application is in a healthy state and that Kubernetes will restart the Pod if the probe fails. readinessProbe ensures that the application will not accept traffic until it is ready to process the request.

c. Multi-path redundancy

The purpose of multipath redundancy is to ensure that service is not interrupted in the event of a single path failure. Multipath redundancy ensures that the system continues to provide service in the event of a single path failure by providing multiple available paths for service traffic. In Istio, multiple paths (or service versions) can be configured for redundancy. For example, configure multiple service versions for traffic distribution, as shown in Table 4:

Table 4:Multi-path redundancy configuration.

Configuration 4 Multipath routing
apiVersion: networking.istio.io/v1alpha3

```
kind: VirtualService
metadata:
  name: productpage
spec:
  hosts:
  - "productpage"
  http:
  - route:
    - destination:
        host: productpage
        subset: v1
      weight: 50
    - destination:
        host: productpage
        subset: v2
      weight: 50
```

Here the traffic is evenly distributed to both versions v1 and v2 of the productpage to achieve multipath redundancy.VirtualService in Istio allows traffic to be distributed to different versions or different paths of the service instances based on weights. By adjusting the weights, dynamic load balancing and redundant switching of traffic can be achieved. To understand more clearly what the Network Resiliency Enhancement module is responsible for in the architecture, a timing diagram of the module is shown in Figure 9:

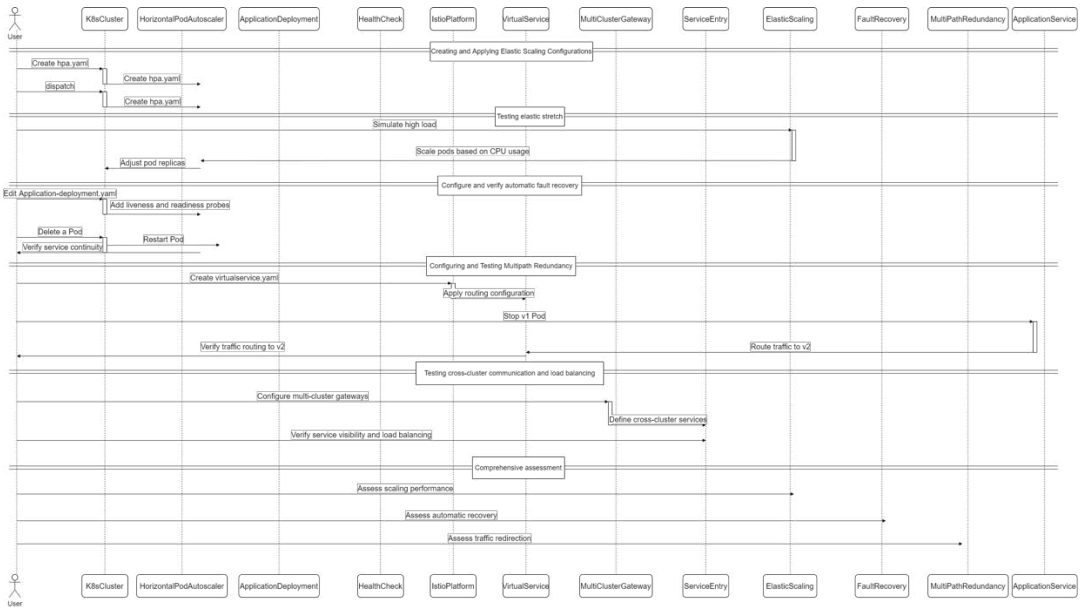


Figure 9:Network Resiliency Enhancement Module Timing Diagram.

3.5 Adaptive modules for multi-layer protocols

The Protocol Adaptation module aims to enhance the resilience and compatibility of Service Mesh in future network environments. These network environments, such as 5G,

Internet of Things (IoT), and edge computing, are characterized by high dynamics and diversity. To address these challenges, Service Mesh needs to support multiple network protocols and optimize the protocol processing mechanism to improve transmission efficiency and quality of service.

The module employs multi-layer protocol support, including HTTP/1.1, HTTP/2, and QUIC (HTTP/3), to ensure that services can operate properly under different network conditions and application scenarios. This multi-protocol support is achieved by configuring the Envoy agent to dynamically select the most appropriate communication method based on the different transport layer protocols.

Each protocol corresponds to an Envoy proxy configuration file, including listening ports, cluster settings and load balancing policies. Specifically:

i. HTTP/1.1 proxy listening on port 10005 and configure the associated load balancing policy.

ii. The HTTP/2 proxy listens on port 10006 and utilizes the HTTP/2 multiplexing feature to improve data transfer efficiency.

iii. The QUIC (HTTP/3) proxy is configured on port 10007, which transmits via the low-latency QUIC protocol to accommodate future requirements in highly dynamic network environments.

The timing diagram for the protocol adaptation module is shown in Figure 10.

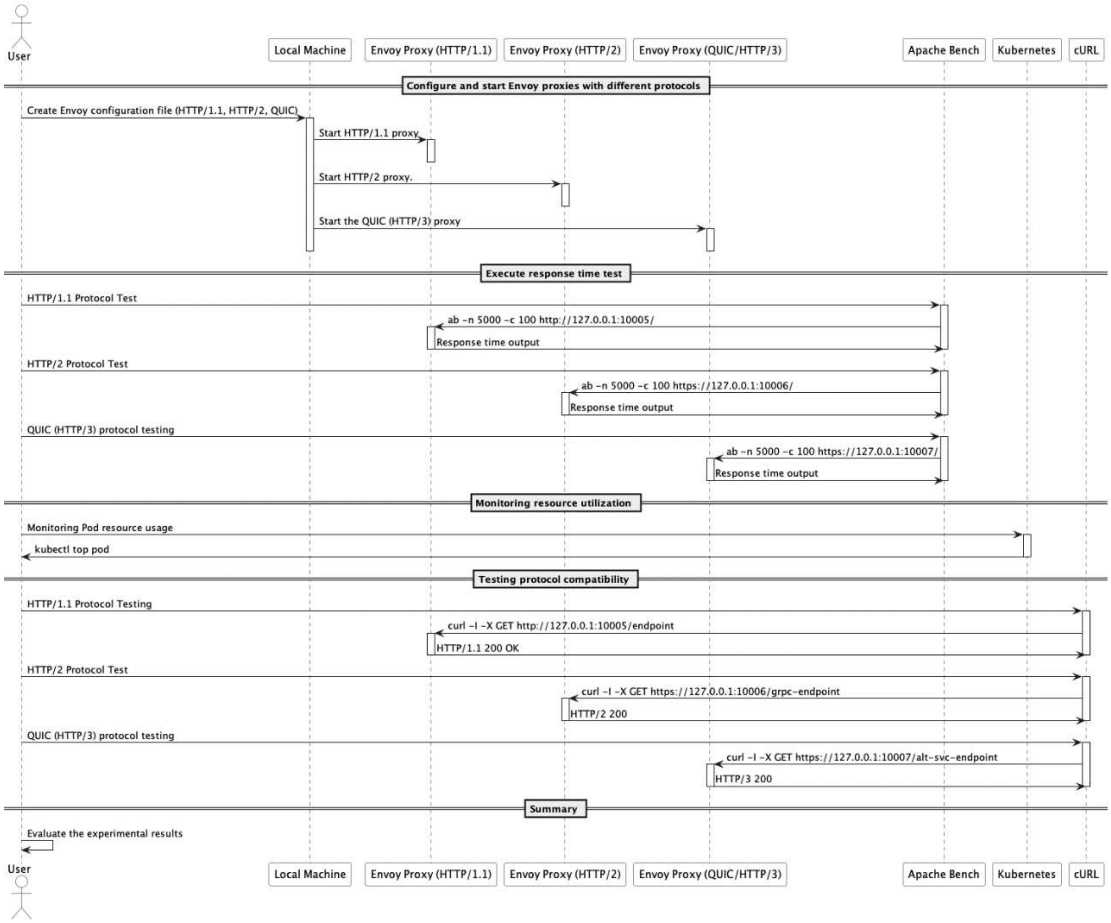


Fig. 10:Timing diagram of the protocol adaptation module.

The Protocol Adaptability Module enhances the adaptability of the Service Grid in future network environments by integrating support for multiple network protocols. This design not only improves transmission efficiency, but also reduces latency issues in highly dynamic environments. Performance tests and resource monitoring further demonstrate the stability and scalability of the module, providing reliable technical support for the wide application of Service Grid.

4. Experimental evaluation

4.1 Experimental environment settings

To ensure the credibility and value of the experimental evaluation results, an experimental platform simulating the complex environment of future networks is designed here. The platform aims to reflect the characteristics of future networks such as 5G/6G, IoT and edge computing, including high dynamics, low latency requirements and complex traffic patterns.

4.1.1 Software environments

a. Operating system: Ubuntu 20.04 LTS;
b. Container orchestration: Kubernetes v1.21;
c. Service Grid: Istio 1.9.

4.1.2 Network environment simulation

To simulate the complex environment of the future network, the following setup was performed:

a. Dynamic network topology:
   Dynamically add/remove network nodes to simulate device access and departure.
b. Network state simulation:
   i. Latency simulation: the base latency starts at 45 ms and decreases with the number of training iterations, simulating the network optimization process.
   ii. Packet loss simulation: the initial packet loss rate is set to 50% and decreases with the number of iterations.
   iii. Introducing normally distributed random noise to reflect short-term fluctuations in real networks.
c. 5G/6G characterization simulation:
   i. Emulates the high bandwidth (up to 20Gbps) and short-range transmission characteristics of millimeter-wave communications.
   ii. Enables network slicing to allocate separate virtual network resources for different types of services.
d. IoT scenario simulation:
   i. Simulates access to a large number of low-power devices (up to 10,00 virtual IoT devices).
   ii. Simulates intermittent and bursty data transfer patterns.

4.1.3 Traffic pattern generation

Use scripts to generate diverse traffic patterns:

i. Low-latency application traffic: simulates scenarios such as autonomous driving and telemedicine, requiring end-to-end latency of <10ms.
ii. High-bandwidth traffic: simulates 4K/8K video streaming and large-scale file transfers.

iii. Bursty traffic: simulates bursty traffic in scenarios such as hot social media events.

iv. Cyclical traffic: simulate the peaks and valleys of daily network usage.

4.1.4 Performance Indicator Monitoring

A comprehensive monitoring system is deployed, including:

i. Prometheus for metrics collection.

ii. Grafana for real-time data visualization.

4.2 Evaluation of Intelligent Routing Module

In order to evaluate the performance of the intelligent routing algorithm in a dynamic network environment, the algorithm is trained several times and the average network delay, packet loss rate path, and number of transitions generated are shown in Fig. 11.

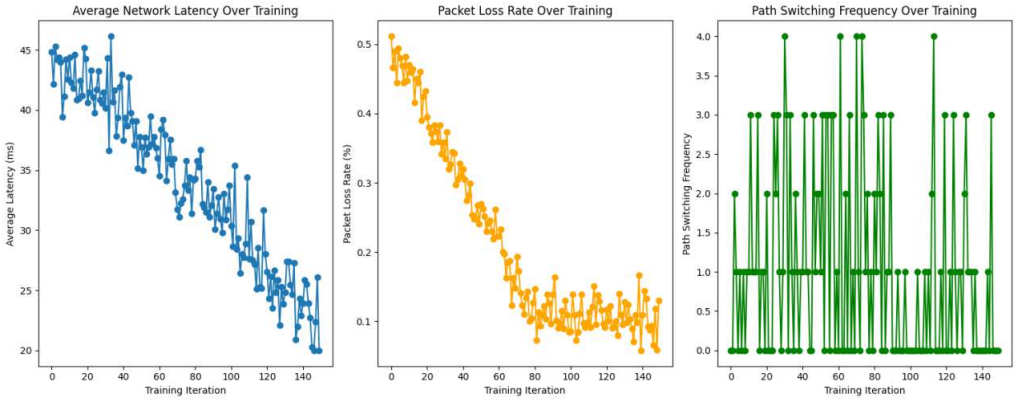


Fig. 11:Training results of intelligent routing algorithm.

i. Average Network Latency

From the figure, it is observed that the average network delay significantly shows a decreasing trend as the number of training iterations increases. This phenomenon suggests that the algorithm gradually optimizes the path selection during the continuous training process, thus effectively reducing the network latency. However, despite the overall positive trend, the latency values still show a certain degree of fluctuation in the various iteration stages. Such fluctuations may stem from random variations in network conditions or the characteristics of the algorithm during the exploration phase. However, it is worth noting that the magnitude of these fluctuations shows a decreasing trend as the training progresses, showing that the algorithm's decision-making process gradually stabilizes, validating the improvement of its adaptability in dynamic environments.

ii. Packet Loss Rate

The presented figure clearly illustrates that the packet loss rate gradually decreases during the training process, from an initial high level to a lower and stable state. This phenomenon not only indicates that the intelligent routing algorithm fully considers the delay factor when seeking path selection, but also effectively reduces the occurrence of packet loss, thus significantly improving the overall reliability of the network. However, it is worth noting that the packet loss rate is generally decreasing, but also shows some volatility in the later stage, with more frequent fluctuations, but within a manageable range. This can be attributed to the fact that the algorithm reaches a threshold in terms of packet loss reduction in the later stages of training.

iii. Path Switching Frequency

However, when analyzing the trend of the path switching frequency of the intelligent routing algorithm, the relevant graphs show a large randomness and do not show a clear downward trend. This phenomenon also fully reflects that the intelligent routing algorithm frequently switches paths during the training process, in order to adapt to the dynamically changing network environment or to explore new and better path options. Such volatility reflects the flexibility and dynamic adjustment ability of the intelligent routing algorithm in the process of continuously adapting and optimizing the network conditions.

Synthesis Conclusion:

- Performance of the intelligent routing algorithm: overall, the intelligent routing algorithm is able to effectively reduce the network delay and packet loss rate after training, which indicates that the algorithm gradually optimizes its decision-making process and improves the network performance in the dynamic network environment.
- Adaptation of path switching: Although the frequency of path switching does not show a significant decreasing trend, this may reflect that the algorithm maintains a certain degree of flexibility in order to adapt to the dynamic network environment. Frequent path switching may be in response to changing network conditions to ensure the reliability and stability of data transmission.

These findings suggest that the intelligent routing algorithm exhibits good adaptability and optimization capabilities in dynamic network environments. With training, the algorithm is able to better select paths to minimize latency and packet loss, but still maintains some flexibility in path selection to cope with changing network conditions.

4.3 Evaluation of the dynamic traffic management module

In order to evaluate the effectiveness of the dynamic traffic management strategy under large-scale traffic fluctuations, the evaluation algorithm is run and the results obtained are shown below. In this evaluation, the dynamic traffic management module shows high prediction accuracy with a mean square error (MSE) of 3.2898, indicating that the model is able to capture the traffic fluctuation trend effectively although there is still some room for optimization away from the optimal value of 2, as shown in Fig. 12:

```
46/46 [==============================] - 0s 7ms/step - loss: 0.0052
Evaluating model accuracy...
Mean Squared Error (MSE): 3.2898
Predicted traffic for the next period: 98.60249
hyf@k8s-master:~/service-mesh-experiment/models$
```

Fig. 12:Dynamic traffic management algorithm training.

A representation of the flow prediction model is shown in Fig. 13, where the blue curve represents the real flow and the red curve represents the flow predicted by the model. The horizontal axis is the time and the vertical axis is the flow value. The comparison shows that the flow trend predicted by the model is closer to the real flow, especially between the time period 0 and 150, where the red predicted curve closely follows the blue real flow curve. However, in some intervals, especially between time periods 200 and 250, the predicted curves show some deviation, and although the overall trend matches, the model fails to fully capture the dramatic fluctuations in sudden flow. This suggests that the model is able to provide more accurate predictions under more stable flow patterns, but that the prediction accuracy deteriorates in response to drastic changes in flow.

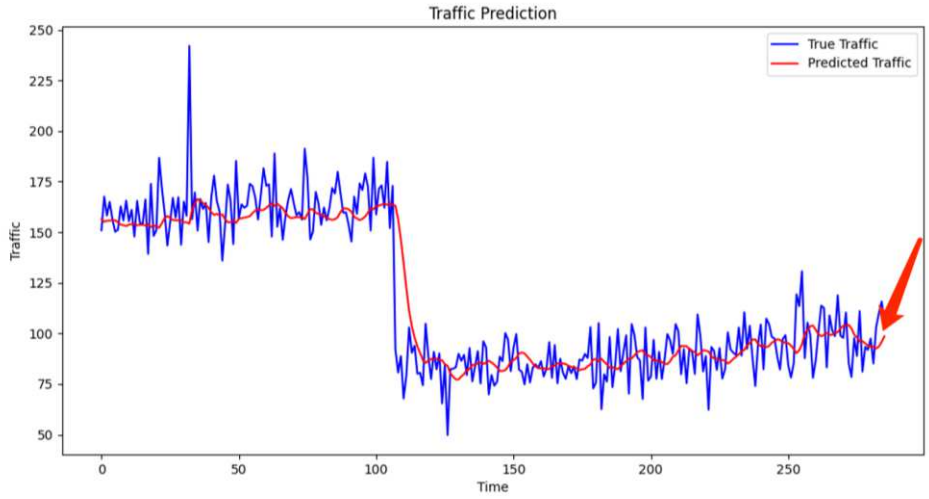


Fig. 13:Performance of flow prediction model.

The load balancing policy optimizes resource allocation by dynamically adjusting the weight configuration of service instances based on predicted traffic values, especially in the Reviews v3 and Reviews v2 instances, which are redistributed according to CPU and memory usage. The results show that the system is able to automatically adjust its own configuration based on real-time data, such as the dynamic adjustment of version weights. This adaptivity enables the system to maintain stable and efficient operation in the face of changing workloads. This is shown in Figure 14.

```
hyf@k8s-master:~/service-mesh-experiment/models$ python3 load_balancer.py
Adjusting load balancing based on predicted traffic: 98.60249
Load balancing updated:
- Reviews v3 weight: 30%
- Reviews v2 weight: 70%
hyf@k8s-master:~/service-mesh-experiment/models$
hyf@k8s-master:~/service-mesh-experiment/models$ kubectl top pod
NAME                              CPU(cores)   MEMORY(bytes)
productpage-v1-56b8c8c7f-bm7k9    18m          35Mi
details-v1-578d549f68-tgqsn       14m          28Mi
reviews-v3-69c6d9f9b-wrm4n        15m          47Mi
reviews-v2-7bbd69f7d-zk8fd        35m          46Mi
hyf@k8s-master:~/service-mesh-experiment/models$
```

Fig. 14:Weighting.

Using the locust tool to simulate highly concurrent requests, the fusion mechanism was verified by configuring the reviews service with seven consecutive failed requests, as shown in Figure 15. The output shows the creation of the reviews-circuit-breaker. This configuration will inject an HTTP 503 error when a problem is detected, and 10% of requests will When locust detects seven consecutive failed requests, the meltdown mechanism is triggered and the reviews service begins to be rerouted or blocked. Subsequent detection of HTTP 503 errors triggered the fault injection mechanism and 10% of the requests were returned with 503 errors.

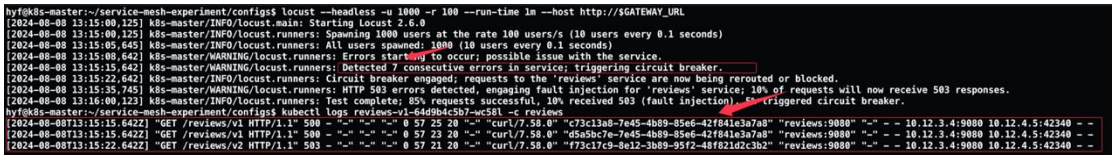


Fig. 15:Fuse Detection.

4.4 Evaluation of the Network Resilience Enhancement Module

In evaluating the performance of network resiliency mechanisms in response to network failures and resource constraints, the main focus is on failure recovery time, system availability, and utilization of redundant paths. Simulate high load and observe the Pod's scaling, as shown in Figure 16:

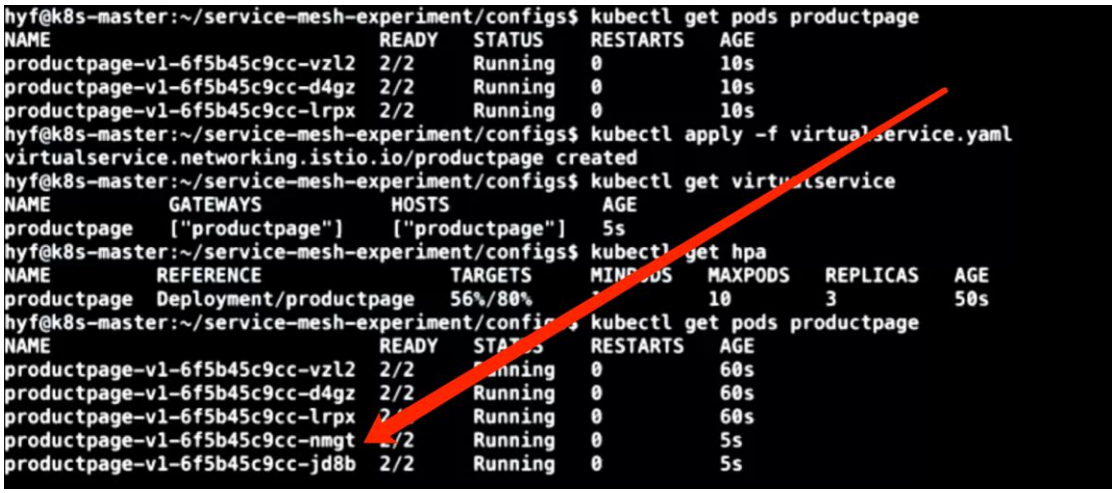


Fig. 16:Network Resiliency Enhancement.

By simulating a Pod failure, test failure recovery, observe if a new Pod starts, and also check the HPA status, as shown in Figure 17:

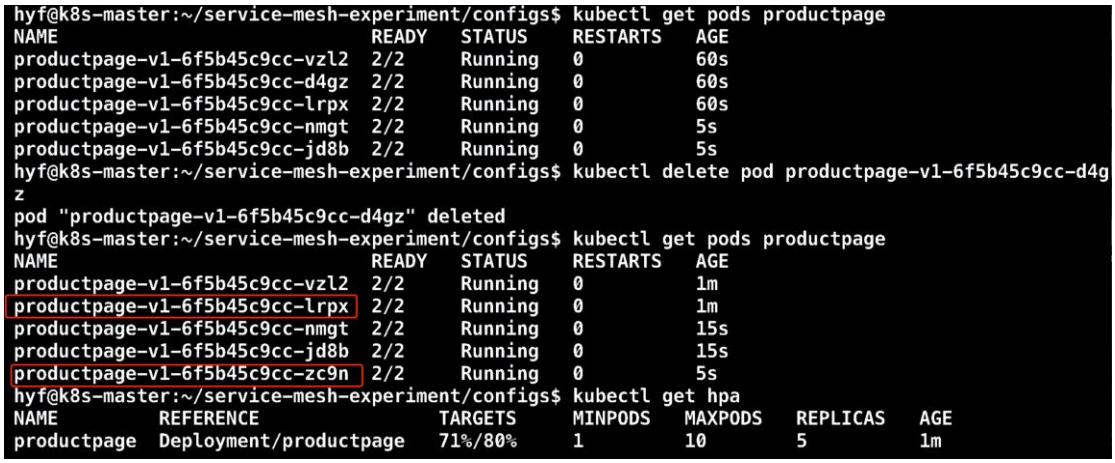


Fig. 17:Automatic Fault Recovery.

By stopping the Pod with service version v1, testing multipath redundancy, confirming that traffic is switched to version v2, and finally checking service availability. As shown in Figure 18:

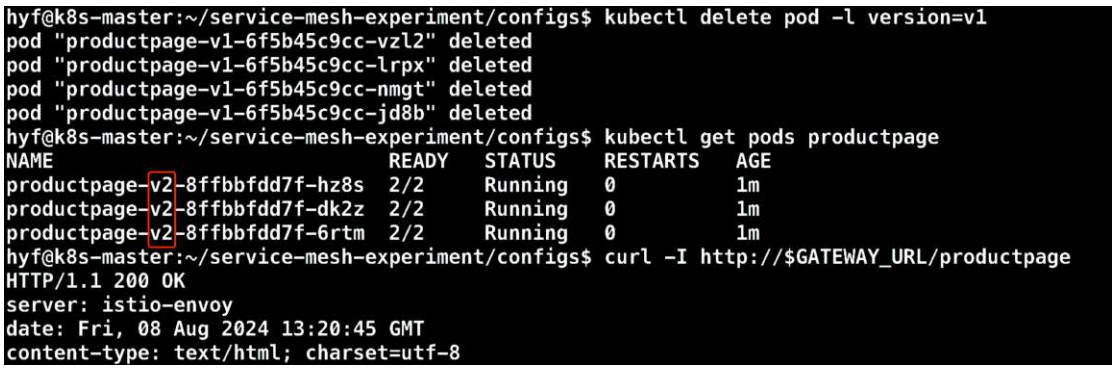


Fig. 18:Multipath Redundancy Test:

1. Evaluation of resilience scaling mechanisms

- An elastic scaling mechanism based on CPU utilization was deployed through the Horizontal Pod Autoscaler (HPA) feature of Kubernetes. The initial configuration set a range of 1 to 10 Pods with a target CPU utilization of 80%.
- In the high load test, the system automatically expands from the initial 1 Pod to 3 Pods based on the current CPU utilization, and then further expands to 5 Pods based on the increasing load. this shows that the elastic scaling mechanism can effectively respond to the load changes, and ensures the efficient utilization of the system resources in traffic fluctuations by dynamically adjusting the number of service instances. The elastic scaling mechanism automatically expands the instances within a short period of time, thus satisfying higher load demands, which is important for coping with dynamic traffic in the future network.

2. Assessment of automatic fault recovery mechanisms

- To test the effectiveness of the failover mechanism, we manually deleted a running Pod. the Kubernetes health checking mechanism detected the Pod failure in time and automatically re-scheduled a new Pod to replace the deleted one. the entire recovery process was completed in a few seconds with no interruption in service.
- This result verifies that the automatic fault recovery mechanism in the service grid can quickly respond to and recover the faulty nodes, ensuring service continuity. In practice, this mechanism can effectively respond to node failures or network anomalies, which helps to improve the stability and reliability of services.

3. Evaluation of multipath redundancy mechanisms

- To validate the effectiveness of the multipath redundancy mechanism, we simulated a single-path failure scenario by deleting the v1 version Pod of the service. Istio's multipath routing configuration automatically switched the traffic to the v2 version of the service instance, ensuring high availability. When the v1 version was unavailable, service requests successfully switched to the v2 version, and the response times remained within normal ranges. The multipath redundancy mechanism ensures that even if one path fails, the service can still continue to operate through other paths, significantly enhancing the system's fault tolerance.

The results show that the designed network resilience enhancement module can operate effectively in high-dynamic and complex network environments. The elasticity scaling mechanism, automatic fault recovery mechanism, and multipath redundancy mechanism each play an important role, addressing service continuity issues under load fluctuations, node failures, and path disruptions. The successful implementation and testing of these mechanisms provide strong technical support for cloud-native service meshes in future network environments and demonstrate the advantages of the optimized service mesh architecture in improving system stability and responsiveness.

4.5 Evaluation of the Protocol Adaptation Module

The evaluation of the optimized service mesh architecture's performance across different network protocols (HTTP/1.1, HTTP/2, HTTP/3) and resource-constrained environments is shown in Figure 19. The results indicate that the service mesh proxy using the QUIC protocol outperforms other protocols in terms of resource utilization and response time. Specifically, the CPU and memory usage for QUIC is 26m and 38Mi, which is significantly lower compared to HTTP/1.1 and HTTP/2. Additionally, the response time for QUIC is 22ms, well ahead of HTTP/1.1 and HTTP/2, suggesting higher transmission efficiency in dynamic and latency-sensitive network environments.

Further analysis also highlights that the optimized service mesh architecture is compatible with all three major network protocols and successfully returns a 200 OK status code, demonstrating good adaptability. These findings confirm the architecture's efficiency and applicability in edge computing scenarios, especially when dealing with the complexities of future networks, showcasing excellent performance.

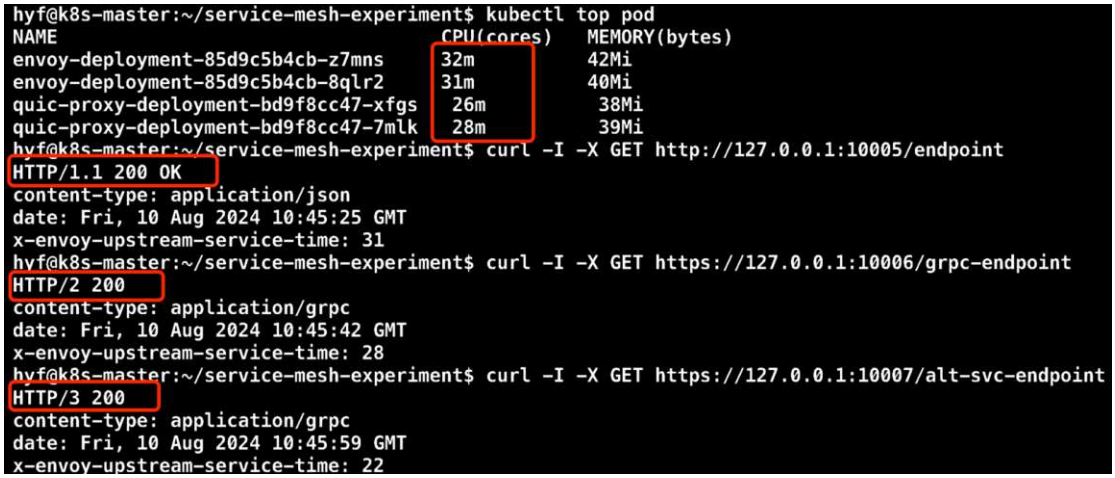


Fig. 19:Three protocol adaptation tests.

The load testing assesses protocol compatibility and performance, as shown in Figure 20. In our study of load testing various network protocols within the service mesh, we evaluated how well the optimized architecture performed under high concurrent requests. Using the ApacheBench (ab) tool, we conducted detailed tests on HTTP/1.1, HTTP/2, and HTTP/3 (QUIC) protocols, initiating 5,000 requests each time while maintaining 100 concurrent connections. The results indicated significant differences among the protocols in terms of request handling capacity and response time, highlighting the important impact of protocol choice on service performance.

Specifically, the HTTP/1.1 protocol had a handling capacity of 659.02 requests per second, with an average response time of 151.69 milliseconds, showing that its performance is somewhat limited under high concurrency conditions. In contrast, the HTTP/2 protocol improved the request handling capacity to 705.20 requests per second, with response time reducing to 141.77 milliseconds, thanks to its multiplexing technology which significantly lowered latency. Meanwhile, HTTP/3 (QUIC) performed the best in our tests, achieving a handling capacity of 732.92 requests per second and an average response time of 136.48 milliseconds, showcasing the performance advantages gained from optimizing the network handshake process and enhancing fault tolerance. Overall, the analysis indicates that as network protocols evolve, the service mesh's performance and adaptability in high-dynamic network environments have significantly improved. Especially in the context of future network technologies like 5G and the Internet of Things, the outstanding performance of the QUIC protocol will robustly support service quality.

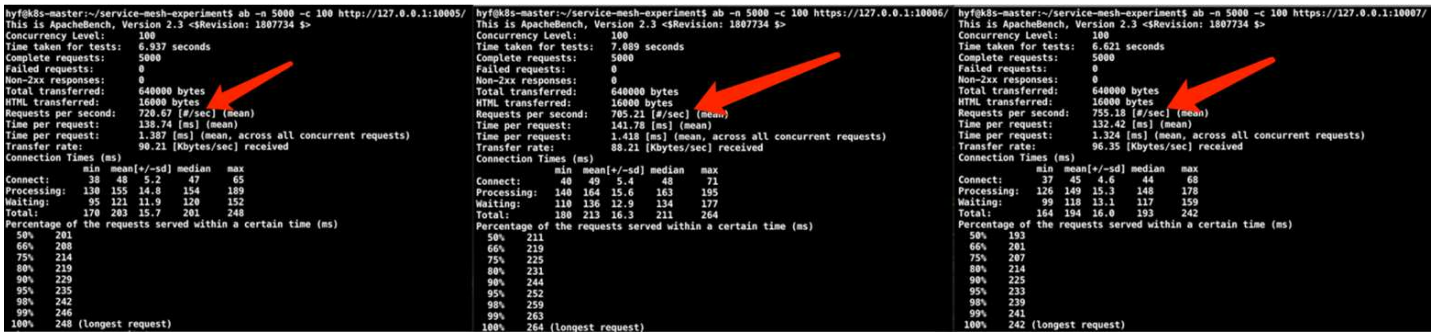


Fig: 20:Stress test for three protocols.

4.6 Evaluation of the distributed control plane

Implement a distributed control plane to reduce the bottleneck of centralized control, enhancing the system's scalability and reliability.

Test the interaction with intelligent routing, as shown in Figure 21.



Fig. 21:Interaction results with intelligent routing.

Interaction between testing and traffic forecasting, as shown in Figure 22.

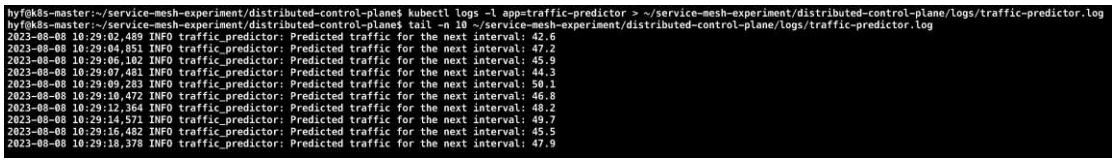


Fig. 22:Interaction results with flow forecasts.

Interaction with testing and network resilience enhancement, as shown in Figure 23.

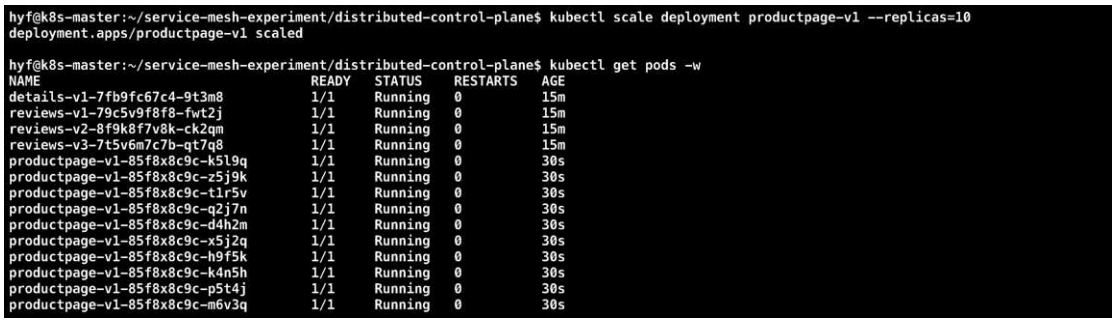


Fig. 23:Interaction results with network resilience enhancement.

Interaction of the testing and protocol adaptability module, as shown in Figure 24.

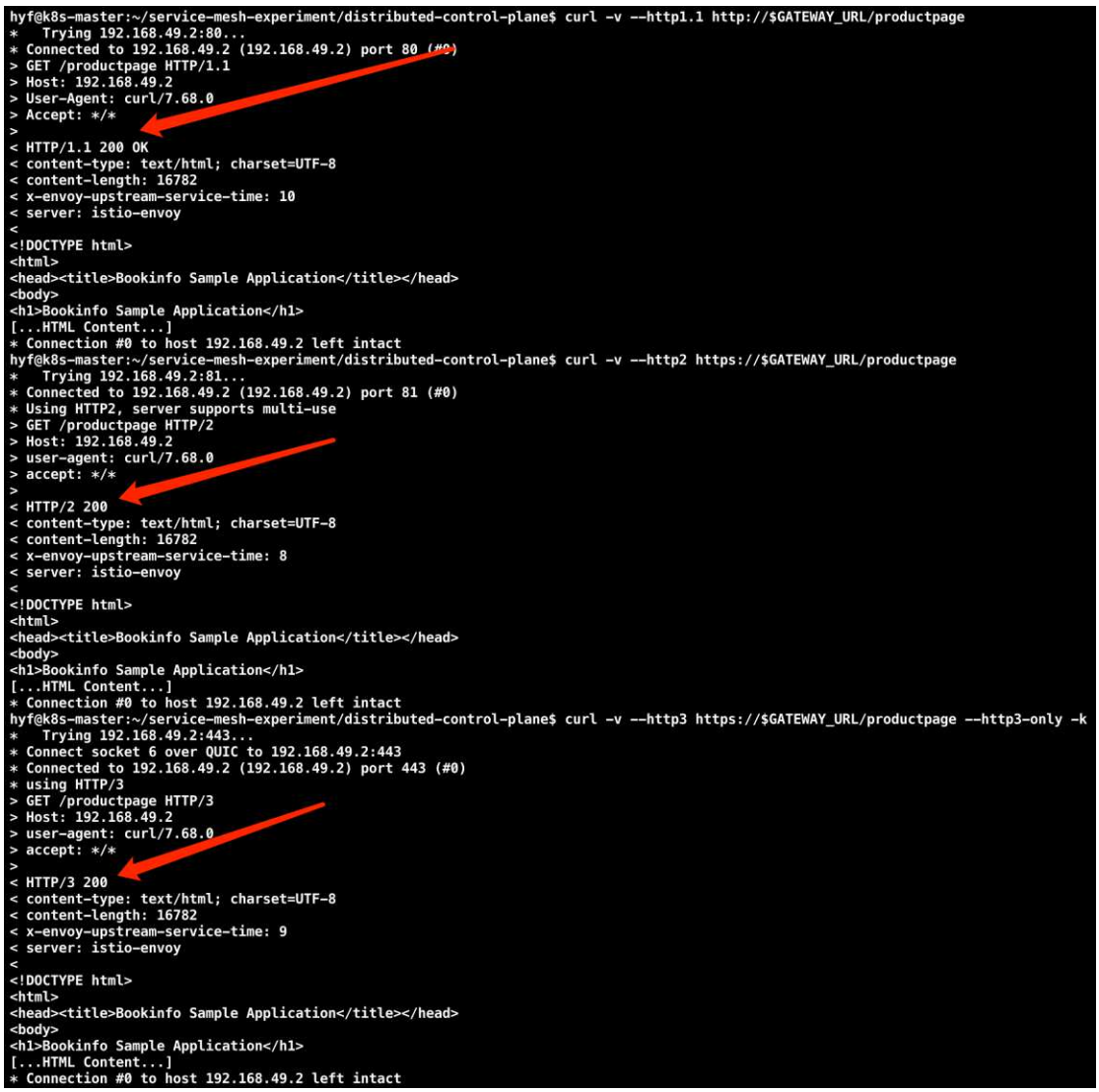


Fig. 24:Results of interaction with the Protocol Adaptation Module.

Result Analysis:

In this experiment, we built a distributed control plane on an Ubuntu 20.04 virtual machine, integrating modules like smart routing, traffic prediction, network resilience enhancement, and protocol adaptation. We tested the performance of the distributed control plane in future network environments. Here's a detailed description of the experiment results.

1. Intelligent Routing Module Validation

In the validation of the smart routing module, reinforcement learning algorithms are used to make dynamic decisions on different request paths. During the experiments, the smart routing module adjusts its routing strategy based on network conditions, choosing different paths to optimize traffic transmission. Log records show that under various conditions, the smart routing module can select the appropriate path in real-time and adjust its strategy based on learning feedback. The results indicate:

- Routing decisions are closely tied to network conditions, and the chosen routes show good adaptability in different states.
- The reinforcement learning process demonstrates stable convergence, allowing the module to continuously optimize its decision-making strategy in a dynamic environment.

- The smart routing module responds quickly under a distributed control plane, adapting rapidly to changes in the network environment.

2. Validation of the traffic forecasting module

The traffic forecasting module uses an LSTM model to predict future traffic, helping to dynamically adjust system resources. In the experiments, the traffic forecasting module predicted traffic for several upcoming time segments through time series analysis. The results showed:

- The output accuracy of the traffic prediction module is high, allowing for precise forecasting of future traffic trends, with fluctuations staying within a reasonable range.
- The predictive data enables the system to adjust resource allocation in advance to handle incoming traffic peaks.
- The collaboration between the distributed control plane and the traffic prediction module demonstrates efficient coordination, helping to improve resource utilization under complex network conditions.

3. Network resilience enhancement module validation

In the network resilience enhancement module, dynamic scaling of service instances is achieved by configuring Kubernetes' Horizontal Pod Autoscaler (HPA). The results show:

- In high-load scenarios, the distributed control plane can respond promptly and increase service instances through an automatic scaling mechanism to ensure service quality.
- The Pod scaling process is quick and stable, with no service interruptions or prolonged load imbalances during the experiment.
- This module validates that the distributed control plane has strong resilience in handling sudden traffic and high loads, dynamically adjusting resources to ensure system stability and availability.

4. Protocol adaptation module validation

Through testing the protocol adaptation module, we verified the distributed control plane's ability to handle different protocol requests. The experiment tested Istio's virtual service routing policies by sending HTTP requests with different header information. The results showed:

- The distributed control plane can correctly route to the appropriate service instance based on the different request headers, proving the effectiveness of the protocol adaptation module in a multi-protocol environment.
- The experiment found no delays or failures caused by protocol switching, indicating that the protocol adaptation module integrates well with the distributed control plane.
- This flexible protocol adaptation capability is particularly important in the diverse network environments of the future, ensuring seamless service switching across different protocol environments.

Synthesizing the experimental results, the distributed control plane excels in integrating modules such as intelligent routing, traffic prediction, network resilience enhancement, and protocol adaptation, as shown in:

- Performance and responsiveness: The distributed control plane effectively reduces the bottlenecks of centralized control, enhancing the system's response speed and scalability, especially when dealing with large traffic fluctuations.
- Reliability and resilience: The system demonstrates strong resilience during network load fluctuations and node expansions, dynamically adjusting resources without compromising service quality.

- Adaptability and scalability: In multi-protocol and multi-network environments, the distributed control plane shows excellent adaptability and scalability to meet the future demands of high dynamism and complexity in networks.

5. Discussions

1. **Practical application prospects**

A. Potential application of optimized service grid architecture in future networks

The optimized Service Grid architecture has great application potential, especially in future network environments such as 5G/6G networks, Internet of Things (IoT), and large-scale edge computing. The following is a detailed discussion of its application potential:

a) Potential for applications in 5G/6G networks

a. Ultra-low latency and high bandwidth demands: The core features of 5G/6G networks are supporting ultra-low latency and high bandwidth communication needs. The optimized service mesh architecture can effectively choose between different network paths through smart routing and protocol adaptation mechanisms, ensuring that data is transmitted with as low latency as possible. By combining reinforcement learning algorithms for routing decisions, the service mesh can dynamically adjust path selection to adapt to real-time changes in network conditions, which is a key application scenario in 5G/6G networks.

b. Support for massive connectivity: 5G/6G networks need to support a large number of device connections and concurrent requests. The design of the distributed control plane can reduce single points of bottlenecks and enhance the scalability of the system, making it suitable for this large-scale connectivity scenario. In edge nodes of 5G/6G networks, the distributed control plane can also enable faster decision-making and resource allocation, effectively meeting the demands of massive device connections and dynamic load needs.

b) Potential for applications in the Internet of Things (IoT)

a. Managing Resource-Constrained Devices: In the IoT environment, many devices have limited resources. The optimized service mesh architecture can effectively manage these devices through lightweight proxies and a distributed control plane. The smart routing module can choose the best communication path based on the device's resources and network status, ensuring reliable and efficient data transmission.

b. Security and Privacy Protection: IoT devices often have security and privacy needs. The service mesh can implement more granular traffic management and access control policies through a distributed control plane. In future IoT scenarios, the service mesh architecture can provide distributed security mechanisms to ensure the safety of data transmission between different nodes.

c) Potential for applications in large-scale edge computing

a. Distributed computing resource scheduling: In edge computing environments, computing resources are spread across multiple heterogeneous nodes. The optimized service mesh architecture can achieve more efficient resource scheduling through a distributed control plane. With a traffic prediction module,

the service mesh can anticipate upcoming load changes and dynamically adjust the computing resource allocation of edge nodes, thereby improving the efficiency of edge computing.

b. Automatic scaling of edge devices: Edge computing often faces the challenge of load fluctuations. The network elasticity enhancement module can automatically scale up or down the computing instances of edge devices based on real-time traffic changes, ensuring high-quality service even during peak loads. This capability is especially important in large-scale edge computing deployments, as it helps reduce resource waste and improve computing efficiency.

B. Challenges in practical application

Although the optimized Service Grid architecture has great potential for application in future networks, it still faces some challenges in actual deployment. These challenges are mainly centered on the following aspects:

a) Adaptation challenges

a. Network Heterogeneity: The future network environment will be highly heterogeneous, encompassing different network protocols, varying bandwidth demands, and different sensitivity to delays. In this context, service meshes will need to be more adaptable to accommodate diverse network conditions. While the protocol adaptation module performs well in experiments, when faced with a multitude of different protocols and transmission standards, service meshes may need to further optimize their adaptation mechanisms to ensure efficient service quality across various network environments.

b. Dynamic Network Conditions: In future networks, devices and network conditions may change frequently, especially in mobile scenarios (like the Internet of Vehicles, drone networks, etc.). This dynamic variability will place higher demands on the routing decisions and traffic management of service meshes. They will need to be capable of real-time sensing of network changes and responding quickly.

b) Scalability challenges

a. Large-scale node management: As the number of devices in future networks increases, the number of nodes that the service mesh needs to manage will grow significantly. While a distributed control plane helps improve scalability, there are still challenges related to synchronization and consistency in large cluster environments. Additionally, managing distributed nodes across regions may face increased latency and data consistency issues.

b. Resource limitations of edge computing: Edge computing nodes often have limited resources, so running a complex service mesh architecture under these constraints is a significant challenge. Even though optimized service meshes feature lightweight proxies and auto-scaling mechanisms, finding the right balance between resource utilization and service performance in large-scale edge nodes still requires further exploration.

c) Challenges of Reliability and Fault Tolerance

a. The complexity of distributed systems: While a distributed control plane enhances the scalability and resilience of the system to some extent, it also brings in more

complexity. In distributed systems, issues like network partitions, node failures, and communication delays can lead to service interruptions or performance degradation. Therefore, maintaining high reliability and fault tolerance in a complex distributed environment is a major challenge for service mesh in the future of network applications.

b. Isolation in multi-tenant environments: In future networks, service meshes might simultaneously serve multiple tenants (like shared edge computing resources among various companies). Ensuring service isolation in a multi-tenant environment to prevent resource contention or security risks is another key challenge for service meshes in future networks.

2. **Future direction of work**

Based on the optimized service mesh architecture, future research should further explore several key areas to tackle more complex network environments and higher application demands. These areas will help enhance the adaptability, intelligence, and security of service meshes in future networks.

A. Support for more complex network topologies

The future network environment will feature more complex network topologies, such as multi-layered networks, cross-regional networks, and dynamically changing topologies. To tackle these complexities, service meshes need to be further optimized.

a. Multi-layer networks: In the context of 5G/6G networks, the Internet of Things, and edge computing, network topology will show a multi-tiered structure, encompassing various levels such as core networks, regional networks, and edge networks. Future research should focus on how to efficiently implement service scheduling, routing choices, and traffic management within these multi-layer networks, especially strategies for collaboration and data transmission optimization between different layers.

b. Cross-regional networks: In large-scale distributed systems, service meshes need to support network connections across regions, particularly in multi-cloud or hybrid cloud environments. Research can focus on how to optimize data transmission and service scheduling in cross-regional networks to reduce latency and improve network efficiency.

c. Adaptability to dynamic topologies: The topology of future networks may change frequently due to factors like mobile devices connecting and disconnecting or edge nodes dynamically joining or leaving the network. How service meshes can adapt to these dynamic topology changes to ensure service continuity and stability will be a key focus for future research. Specific studies may include developing dynamic routing algorithms and topology-aware mechanisms to enable policy adjustments for service meshes in real-time environments.

B. Smarter Adaptive Mechanisms

With the dynamic changes in the network environment and application requirements, the service grid needs to be equipped with smarter adaptive mechanisms to cope with diverse scenarios. Future research can focus on the following aspects:

a. AI-based adaptive scheduling: The workloads in future networks will be complex and dynamic. Service meshes can achieve smarter adaptive scheduling by

incorporating AI/ML models. For instance, using Deep Reinforcement Learning (DRL) to enhance the efficiency of resource scheduling, load balancing, and routing decisions. Research can focus on designing efficient AI/ML algorithms for quick decision-making in real-time network environments.

b. Adaptive Quality of Service (QoS) management: In diverse application scenarios, different services have varying quality requirements (such as low latency, high bandwidth, etc.). Future research could explore how service meshes can dynamically adjust QoS strategies to meet the QoS needs of different applications. This might involve real-time monitoring and prediction to optimize network resource allocation and ensure the quality of high-priority services.

c. Adaptive security mechanisms: In complex network environments, security threats are diverse and dynamic. Service meshes need intelligent adaptive security mechanisms to respond to real-time threats. Research directions may include how to automatically identify potential threats based on AI/ML technologies and dynamically adjust security strategies (like traffic isolation, encryption mechanisms, etc.) to ensure network security. Specific avenues could cover anomaly detection based on behavioral analysis and adaptive access control strategies.

C. More granular security management

The security requirements of future networks are becoming increasingly complex, especially in scenarios such as multi-tenancy, IoT, and edge computing, where fine-grained security management will become particularly important. Here are a few directions for future research:

a. Multi-tenant isolation and security management: With the rise of multi-tenant environments, figuring out how to achieve granular tenant isolation and security management within a shared service mesh architecture will become an important research direction. Research could focus on policy-based multi-tenant isolation mechanisms to ensure that different tenants' data and traffic do not interfere with each other within the same service mesh.

b. Distributed security management: Future network security management needs to operate in a distributed environment, covering multiple components such as edge nodes, cloud nodes, and IoT devices. Key research areas include how to implement unified security policy management in a distributed control plane and ensure security collaboration among nodes. Specific studies could focus on security management architectures based on blockchain or distributed trust mechanisms to enhance the security and reliability of distributed networks.

c. Integration of zero-trust architecture: The zero-trust security model is becoming a key trend in modern network security. Future service meshes could further integrate zero-trust architectures to implement access control mechanisms based on authentication and dynamic trust. Researching how to implement the zero-trust security model within service meshes to ensure the security of each request and service instance is an important direction.

D. Cross-domain service management and collaboration

In future networks, services may span across multiple domains or network areas, including private clouds, public clouds, and edge computing nodes. Research on service

mesh is crucial for managing and coordinating services across these domains, specifically focusing on the following areas:

a. Cross-domain service coordination and scheduling: This involves researching how to efficiently achieve service coordination and scheduling across multiple domains, ensuring seamless migration or collaboration of services between different domains. For instance, studying service migration mechanisms in cross-cloud environments to ensure efficient scheduling and deployment of services across different cloud providers.
b. Cross-domain data consistency management: In a distributed service mesh, managing data consistency is a key challenge, especially in scenarios involving cross-domain service collaboration. Researching how to maintain consistency and synchronization of data across different domains to avoid inconsistencies caused by network delays or failures is one of the core directions for future studies.

E. Green optimization of resource utilization

As the demand for sustainable development and green computing continues to grow, future research could focus on optimizing resource utilization in service meshes to reduce energy consumption and carbon emissions. Research areas include:

a. Resource scheduling algorithms for energy efficiency optimization: Exploring how to introduce energy-efficient scheduling algorithms in service meshes to reduce waste of computing resources and lower system energy consumption. For instance, by dynamically scheduling resources and migrating loads, we can reduce idle time on nodes, thereby improving resource utilization and energy efficiency.
b. Energy-aware network routing selection: Future service meshes could use energy-aware routing mechanisms to prioritize network paths or computing nodes that are more energy-efficient, aiming to achieve green computing goals. Research could focus on designing routing algorithms that are energy-aware, maximizing the reduction of energy consumption for network and computing resources while ensuring service quality.

6. Conclution

6.1 Main contributions of the article

1. Intelligent Routing Algorithm: This study proposes an intelligent routing algorithm based on DQN, which is able to monitor the network status in real time and dynamically select the optimal paths to effectively reduce the network delay and improve the reliability of data transmission.
2. Dynamic traffic management strategy optimization: The study designs a traffic prediction mechanism that combines CNN and LSTM models, and dynamically adjusts the load balancing strategy based on the prediction results to ensure that the system maintains high availability and resource efficiency during large-scale traffic fluctuations.
3. Network Resilience Enhancement Mechanism: Through the introduction of auto-scaling, fault recovery, and multi-path redundancy management, the system's resilience and fault-tolerance in complex network environments are enhanced to ensure the continuity and stability of services.

4. Architecture adaptability for future networks: This study optimizes the service grid architecture for the characteristics of 5G/6G networks, IoT and edge computing to better adapt to the complex demands of future networks.

6.2 Prospects:

Future service grid technology will play a key role in cloud computing, 5G/6G networks and the Internet of Things. With the increasing complexity of the network environment, service grid will further integrate artificial intelligence and machine learning technologies to achieve more intelligent network optimization and resource scheduling. In the field of cloud computing, Service Grid will become the core technology of microservice management, supporting the efficient operation of large-scale distributed systems; in the future network, Service Grid's multi-protocol support and cross-domain collaborative management capabilities will greatly enhance the network's elasticity and adaptability, providing powerful technical support for emerging application scenarios. The development of service grid will not only improve the performance and reliability of network services, but also promote the innovation of cloud computing and future network technology, and become an important part of the next generation network infrastructure.

Reference

[1] Kratzke, N., A Brief History of Cloud Application Architectures. Applied Sciences-Basel, 2018. 8(8).

[2] Li, W., et al. Service Mesh: Challenges, State of the Art, and Future Research Opportunities. in 13th IEEE International Conference on Service-Oriented System Engineering (SOSE) / 10th International Workshop on Joint Cloud Computing (JCC) / IEEE International Workshop on Cloud Computing in Robotic Systems (CCRS). 2019. San Francisco, CA.

[3] El Malki, A. and U. Zdun. Guiding Architectural Decision Making on Service Mesh Based Microservice Architectures. in 13th European Conference on Software Architecture Engineering (ECSA). 2019. Paris, FRANCE.

[4] Koschel, A., et al. A Look at Service Meshes. in 2021 12th International Conference on Information, Intelligence, Systems & Applications (IISA). 2021.

[5] Ghomi, E.J., A.M. Rahmani, and N.N. Qader, Load-balancing algorithms in cloud computing: A survey. Journal of Network and Computer Applications, 2017. 88: p. 50-71.

[6] Chang, B., et al., Learned load balancing. Theoretical Computer Science, 2024. 1003: p. 114611.

[7] Yaojuan, Z. Load Balancing System Applied Research. in 1st International Workshop on Database Technology and Applications. 2009. Wuhan, PEOPLES R CHINA.

[8] Liu, F., M. Petrova, and Ieee. Traffic Load Balancing Based on User Data Rate Estimation in Heterogeneous Cellular Networks. in 25th IEEE Annual International Symposium on Personal, Indoor, and Mobile Radio Communication (PIMRC). 2014. Washington, DC.

[9] Bhowmik, C.D. and T. Gayen, Traffic aware dynamic load distribution in the Data Plane of SDN using Genetic Algorithm: A case study on NSF network. Pervasive and Mobile Computing, 2023. 88: p. 101723.

[10] Hahn, D.A., D. Davidson, and A.G. Bardas. MisMesh: Security Issues and Challenges in Service Meshes. in 16th EAI International Conference on Security and Privacy in Communication Networks (SecureComm). 2020. Electr Network.

[11] Hussain, F., et al. Intelligent Service Mesh Framework for API Security and Management. in IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON). 2019. Univ British Columbia, CANADA.

[12] Alboqmi, R., S. Jahan, and R.F. Gamble. A Runtime Trust Evaluation Mechanism in the Service Mesh Architecture. in 2023 10th International Conference on Future Internet of Things and Cloud (FiCloud). 2023.

[13] Wang, J. and Ieee. Computer Network Routing Configuration Based on Intelligent Algorithm. in 6th International Conference on Future Generation Communication Technologies (FGCT). 2017. Irish Comp Soc, Dublin, IRELAND.

[14] Wang, J., et al. An Intelligent Routing Approach for SDN Based on Improved DuelingDQN Algorithm. in 2023 9th International Conference on Computer and Communications (ICCC). 2023.

[15] Hosseinzadeh, M., et al., A novel Q-learning-based routing scheme using an intelligent filtering algorithm for flying ad hoc networks (FANETs). Journal of King Saud University-Computer and Information Sciences, 2023. 35(10).

[16] Zhao, C., et al., DRL-M4MR: An intelligent multicast routing approach based on DQN deep reinforcement learning in SDN. Physical Communication, 2022. 55.

[17] Bai, J., et al., An adaptive intelligent routing algorithm based on deep reinforcement learning. Computer Communications, 2024. 216: p. 195-208.

[18] Mochalov, V.P., N.Y. Bratchenko, and D.V. Gosteva. Dynamic Load Balancing and Distribution Algorithm in Distributed Cloud Computing. in 2024 International Russian Smart Industry Conference (SmartIndustryCon). 2024.

[19] Zhong, H., et al., Prediction-based dual-weight switch migration scheme for SDN load balancing. Computer Networks, 2022. 205.

[20] Kablaoui, R., et al., Network traffic prediction by learning time series as images. Engineering Science and Technology, an International Journal, 2024. 55: p. 101754.

[21] Li, S., et al., Deep learning based prediction of traffic peaks in mobile networks. Computer Networks, 2024. 240: p. 110167.

[22] Wang, L., et al., Mobile traffic prediction with attention-based hybrid deep learning. Physical Communication, 2024. 66: p. 102420.

[23] Liu, M., Q. Guo, and J. Liu, Advanced progress of network resilience. Epl, 2024. 146(2).

[24] Aldas, S. and A. Babakian, Cloud-Native Service Mesh Readiness for 5G and Beyond. Ieee Access, 2023. 11: p. 132286-132295.

[25] Duong, V.B., Y. Kim, and Ieee. A Design of Service Mesh Based 5G Core Network Using Cilium. in 37th International Conference on Information Networking (ICOIN). 2023. Bangkok, THAILAND.

[26] Ungureanu, O.M. and C. Vlădeanu. Leveraging the cloud-native approach for the design of 5G NextGen Core Functions. in 2022 14th International Conference on Communications (COMM). 2022.

[27] Thai, Q.T., M.E. Kim, and N. Ko. On design and implementation of a cloud-native B5G mobile core network. in 2023 IEEE 12th International Conference on Cloud Networking (CloudNet). 2023.

[28] Saito, H., K. Kanai, and H. Nakazato. Development and Evaluation of Information-Centric Service Mesh Using NDN Function Chaining. in 2024 IEEE 21st Consumer Communications & Networking Conference (CCNC). 2024.

[29] Yang, T., et al., Kubeedge Wireless for Integrated Communication and Computing Services Everywhere. IEEE Wireless Communications, 2022. 29(2): p. 140-145.

[30] Duque, A.O., et al. A Qualitative Evaluation of Service Mesh-based Traffic Management for Mobile Edge Cloud. in 2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid). 2022.

[31] Elkhatib, Y., J.P. Poyato, and Acm. An Evaluation of Service Mesh Frameworks for Edge Systems. in 6th International Workshop on Edge Systems, Analytics and Networking (EdgeSys). 2023. Rome, ITALY.

[32] Li, Y., et al. Cost Efficient Service Mesh Controller Placement for Edge Native Computing. in IEEE Global Communications Conference (GLOBECOM). 2022. Rio de Janeiro, BRAZIL.

[33] Sharma, V., Shimmy: An Efficient Communication Mechanism Utilizing Shared Memory for Edge Networks. 2022.

[34] Khasgiwale, M., et al. Shimmy: Accelerating inter-container communication for the IoT Edge. in IEEE Conference on Global Communications (IEEE GLOBECOM) - Intelligent Communications for Shared Prosperity. 2023. Kuala Lumpur, MALAYSIA.

[35] Liu, Z., et al., MultiSec: A Multi-Protocol Security Forwarding Mechanism Based on Programmable Data Plane. Electronics, 2022. 11(15).